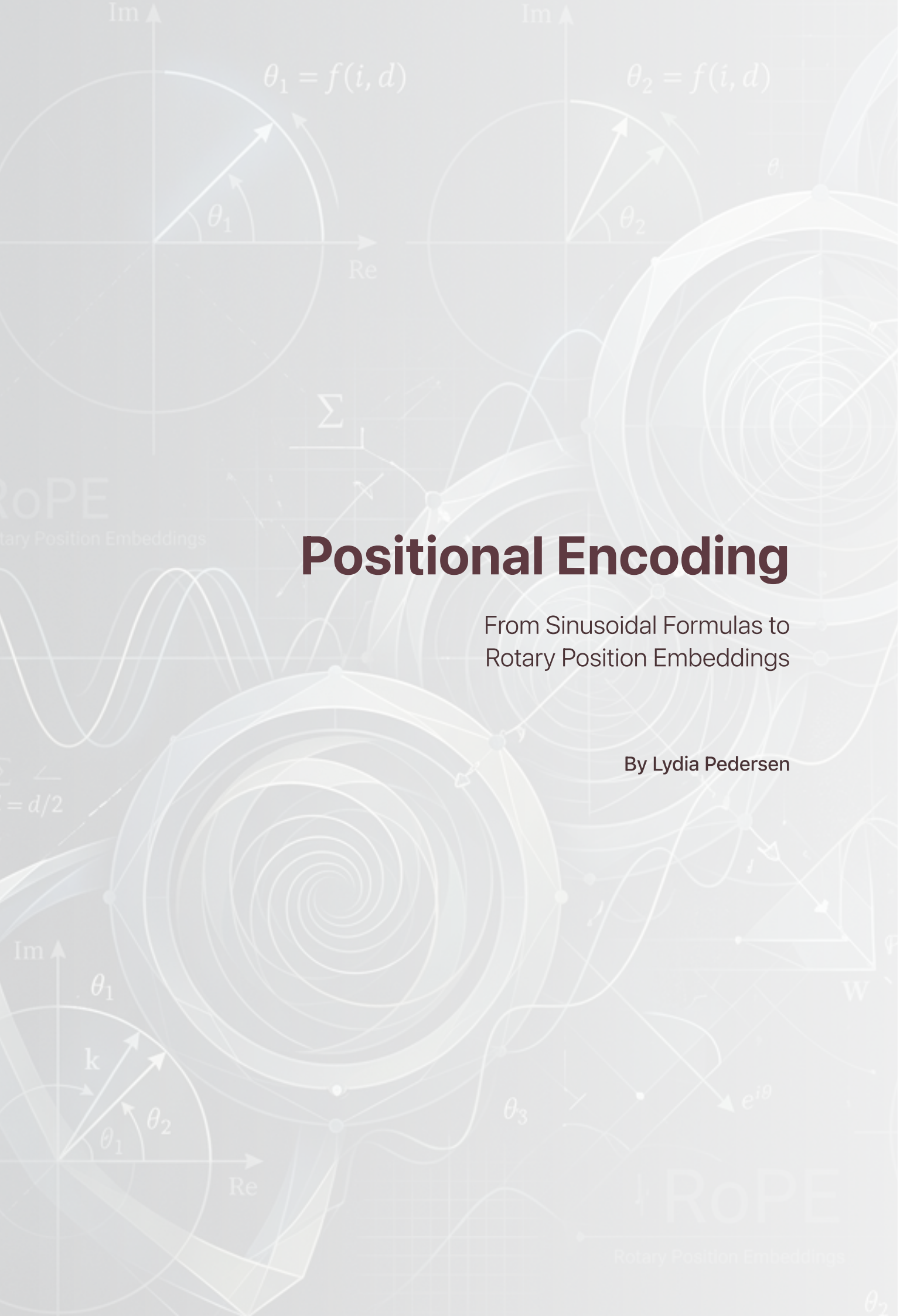

Positional Encoding

From Sinusoidal Formulas to
Rotary Position Embeddings

By Lydia Pedersen

Positional Encoding: From Sinusoidal Formulas to Rotary Position Embeddings

Table of Contents

Preface: The Gap Between Static Geometry and Sequential Structure

The preceding series on Embeddings terminates with an embedding tensor $X \in \mathbb{R}^{T \times d_{model}}$ whose rows encode the distributional signatures of individual words. Each row is a dense, continuous vector shaped by billions of gradient updates into a point whose position in the embedding space reflects the statistical patterns of its token across the training corpus. But this tensor is order-agnostic: the sequences `The` `quick` `brown` `fox` and `fox` `brown` `quick` `The` produce the same set of embedding vectors, merely arranged in different rows. The embedding lookup treats each row independently and has no mechanism to encode which word appeared at which position.

The subsequent series on Transformers requires a tensor that encodes both meaning and position. The self-attention mechanism, which allows each token to attend to every other token in the sequence, is itself a set operation on unordered vectors unless positional information has been injected beforehand. Without that injection, the Transformer cannot distinguish `dog` `bites` `man` from `man` `bites` `dog`.

This series derives the mathematical mechanisms that bridge that gap.

The Scope

The derivation proceeds through two distinct positional encoding paradigms, each worked through the same concrete toy model:

- **Vocabulary ($V = 8$):** `The` `quick` `brown` `fox` `jumps` `over` `lazy` `dog`
- **Embedding Dimension (d_{model}):** 3 dimensions.
- **Toy sequence ($T = 4$):** `The` `quick` `brown` `fox`

The first paradigm is the **sinusoidal absolute positional encoding** introduced by Vaswani et al. (2017) in the original Transformer paper. This mechanism assigns a fixed, deterministic vector to each position in the sequence and adds it element-wise to the embedding vector. It is the historical starting point: the first published solution to the problem of injecting order into a set-based architecture. The series derives the sinusoidal formula from its design constraints, computes every entry of the positional encoding matrix, and demonstrates the addition operation that produces the positionally-enriched tensor.

The second paradigm is **Rotary Position Embeddings** (RoPE), introduced by Su et al. (2021). As of 2026, RoPE is the positional encoding mechanism used in every major frontier language model, including GPT-4, Claude, Gemini, Llama 3, DeepSeek, Mistral, and Qwen. Rather than adding a positional vector to the embedding, RoPE applies a position-dependent rotation to the query and key vectors inside the attention mechanism, ensuring that the dot product between any two positions depends only on their relative offset. The series derives RoPE from its complex-number foundations, computes every rotation matrix, and proves the relative-distance property that makes it the dominant mechanism in production.

The sinusoidal derivation is not wasted effort. It establishes the trigonometric vocabulary, the rotation interpretation, and the relative-distance intuition that RoPE generalizes. The two paradigms share a frequency base, and the mathematical progression from additive encoding to rotational encoding is itself the central narrative of the series.

The Transformers series (which follows this one in the pipeline) includes a brief treatment of sinusoidal positional encoding using a different toy model ($V = 12$, $d_{model} = 6$). Readers who complete both series will recognize the identical mathematical structure applied at different scales.

The Architecture

The series follows the progression from the order-agnostic embedding tensor to the position-aware representations consumed by the Transformer:

1. **The Permutation Invariance Problem.** The pairwise dot-product similarity matrix $S = XX^\top$ is computed for the toy sequence and shown to be invariant under row permutations: reordering the sequence changes the labels but not the similarity scores. The embedding tensor carries no positional information.
2. **Design Constraints and the Sinusoidal Formula.** Five mathematical constraints (uniqueness, boundedness, determinism, smoothness, relative representability) are enumerated, and the sinusoidal positional encoding formula is derived as the function satisfying all five. Every entry of the 4×3 positional encoding matrix is computed from the formula.
3. **Element-Wise Addition and the Positionally-Enriched Tensor.** The sinusoidal encoding is added element-wise to the embedding tensor, producing $X_{pos} = X + PE$. The similarity matrix is recomputed and shown to break permutation invariance. The structural limitations of additive absolute encoding (single-layer injection, absolute position labels, length extrapolation degradation) are identified.
4. **The Query-Key Framework and the Relative Position Requirement.** The minimal query-key dot-product mechanism is introduced at the scope required by RoPE: learned projection matrices W_Q and W_K map embeddings to query and key vectors whose dot product determines relevance scores. The relative position requirement is stated formally.
5. **Rotary Position Embeddings: Deriving the Rotation Matrix.** The RoPE operation is derived from its complex-number foundations. Each dimension pair of the query and key vectors is represented as a complex number and rotated by a position-dependent angle. The 2×2 rotation blocks are expanded into the full block-diagonal rotation matrix, and the angular frequency for the toy model is computed.
6. **Rotary Position Embeddings: Computing the Rotated Vectors.** The rotation matrix is evaluated at every position in the toy sequence, producing four distinct rotation matrices R_0 through R_3 . These matrices are applied to every query and key vector from Chapter 4, producing the rotated vectors consumed by the attention mechanism.
7. **The Relative-Distance Property of RoPE.** The central algebraic identity $R_m^\top R_n = R_{n-m}$ is proved from the angle-addition formulas for sine and cosine. The identity is verified numerically using the rotated vectors from the previous chapter: the dot product of rotated queries and keys is shown to depend only on relative position offset.
8. **The Modern Landscape and Frontier Extensions.** Learned absolute PE, relative PE (Shaw et al., 2018), ALiBi (Press et al., 2022), iRoPE (Meta, 2025), and RoPE long-context extensions (YaRN, NTK-aware scaling) are surveyed. Each mechanism is explained through its motivation, its conceptual operation, and the shortcomings that drove the field toward its successor, tracing the evolutionary arc that culminated in RoPE's dominance.

Every trigonometric identity is derived explicitly. Every rotation matrix is computed entry by entry. Every abstract result is grounded in the toy vocabulary. The mathematics speaks for itself.

Chapter 1: The Permutation Invariance Problem

The preceding Embeddings series terminates with an embedding tensor $X \in \mathbb{R}^{T \times d_{model}}$ whose rows are the distributional signatures of individual tokens. For the toy sequence `The` `quick` `brown` `fox`, the embedding tensor (established in Embeddings Chapter 7) is:

$$X = \begin{bmatrix} 0.1 & -0.4 & 0.2 \\ 0.5 & 0.1 & -0.8 \\ -0.3 & 0.9 & 0.4 \\ 0.2 & -0.2 & 0.1 \end{bmatrix}_{4 \times 3}$$

Row 0 is the embedding vector for `The`, row 1 is `quick`, row 2 is `brown`, and row 3 is `fox`. Each row was extracted from the embedding matrix W_E by a one-hot lookup that depends only on the token's vocabulary index, not on its position in the sequence. This chapter formalizes a consequence of that independence: the embedding tensor carries no information about word order.

Permutation Matrices

A **permutation matrix** $P \in \mathbb{R}^{T \times T}$ is a square matrix in which every row and every column contains exactly one entry equal to 1, with all other entries equal to 0. Multiplying P by a matrix X on the left rearranges the rows of X according to the permutation encoded in P, without altering the row vectors themselves.

A 2×2 permutation matrix that swaps two rows:

$$P_{swap} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

For a sequence of length $T = 4$, a permutation matrix is a 4×4 binary matrix. The entry $P_{ij} = 1$ indicates that row i of the product PX receives row j of the original matrix X. The identity matrix I_4 is the trivial permutation: it leaves all rows in place.

$$I_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The operation PX does not modify the embedding vectors. It rearranges which vector appears at which row index. If P encodes a reversal, then PX places the last row of X first, the second-to-last row second, and so on. The vectors themselves (the three-dimensional embedding coordinates) are unchanged.

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \text{row } 0 \\ \text{row } 1 \\ \text{row } 2 \\ \text{row } 3 \end{bmatrix} = \begin{bmatrix} \text{row } 3 \\ \text{row } 2 \\ \text{row } 1 \\ \text{row } 0 \end{bmatrix}$$

The Pairwise Dot-Product Similarity Matrix

The dot product between two embedding vectors measures their geometric similarity: the extent to which the two vectors point in the same direction in $\mathbb{R}^{d_{model}}$. The **pairwise dot-product similarity matrix** $S = XX^\top \in \mathbb{R}^{T \times T}$ contains the dot product between every pair of rows in X.

In a Transformer architecture, the core mechanism for processing a sequence is self-attention, which computes relevance scores via dot products between token representations. The matrix S represents the raw geometric relationship between every token in the sequence before any learned projections are applied. Computing this matrix isolates the baseline semantic relationships within the embedding tensor. If this matrix remains fundamentally unchanged when the input sequence is reordered, it proves that the self-attention mechanism cannot distinguish word order without external positional injection.

The notation $\in \mathbb{R}^{T \times T}$ is read as "is an element of the set of all $T \times T$ matrices of real numbers". The symbol $\in$ denotes set membership, and $\mathbb{R}$ denotes the set of real numbers. The matrix S is generated by multiplying the $T \times d_{model}$ matrix X by its $d_{model} \times T$ transpose $X^\top$. For the toy sequence ($T = 4$ and $d_{model} = 3$), this operation takes the form:

$$S = \begin{bmatrix} X_{00} & X_{01} & X_{02} \\ X_{10} & X_{11} & X_{12} \\ X_{20} & X_{21} & X_{22} \\ X_{30} & X_{31} & X_{32} \end{bmatrix} \begin{bmatrix} X_{00} & X_{10} & X_{20} & X_{30} \\ X_{01} & X_{11} & X_{21} & X_{31} \\ X_{02} & X_{12} & X_{22} & X_{32} \end{bmatrix}$$

To isolate a single entry S_{ij} in the resulting matrix, row i of the first matrix (X) is multiplied by column j of the second matrix ($X^\top$). Because the columns of $X^\top$ are identical to the rows of X, this operation is equivalent to the dot product of row i and row j.

By standard mathematical convention, all standalone vectors are defined as column vectors (dimensions $d_{model} \times 1$). Because the embedding matrix X has dimensions $T \times d_{model}$, its individual rows are $1 \times d_{model}$ row vectors. Therefore, if the individual token embeddings are the column vectors $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$, then row i of the matrix X is written as the transposed vector $\mathbf{x}_i^\top$.

This convention prevents dimensional mismatches. To compute a scalar dot product via matrix multiplication, a $1 \times d_{model}$ row vector must be multiplied by a $d_{model} \times 1$ column vector. If $\mathbf{x}_i$ and $\mathbf{x}_j$ were defined as row vectors, their dot product would be written as $\mathbf{x}_i \mathbf{x}_j^\top$. Because they are defined as column vectors, the entry S_{ij} is expressed as the dot product $\mathbf{x}_i^\top \mathbf{x}_j$:

$$S_{ij} = \mathbf{x}_i^\top \mathbf{x}_j = \begin{bmatrix} X_{i0} & X_{i1} & X_{i2} \end{bmatrix} \begin{bmatrix} X_{j0} \\ X_{j1} \\ X_{j2} \end{bmatrix}$$

The dot product is executed by multiplying the corresponding elements of the two vectors and summing the results across all dimensions. This mathematical operation produces the summation formula:

$$S_{ij} = \sum_{k=0}^{d_{model}-1} X_{ik} \cdot X_{jk}$$

The resulting matrix S has 16 total positions (4×4). However, the dot product is commutative ($\mathbf{x}_i^\top \mathbf{x}_j = \mathbf{x}_j^\top \mathbf{x}_i$), which guarantees that $S_{ij} = S_{ji}$. The matrix is symmetric across its main diagonal. It contains 4 diagonal entries (representing self-similarity) and 12 off-diagonal entries that form identical pairs across the diagonal. Consequently, there are only 10 unique elements (4 diagonal entries plus 6 unique off-diagonal pairs) that require computation:

$$S = \begin{bmatrix} \mathbf{x}_0^\top \mathbf{x}_0 & \mathbf{x}_0^\top \mathbf{x}_1 & \mathbf{x}_0^\top \mathbf{x}_2 & \mathbf{x}_0^\top \mathbf{x}_3 \\ \mathbf{x}_1^\top \mathbf{x}_0 & \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 & \mathbf{x}_1^\top \mathbf{x}_3 \\ \mathbf{x}_2^\top \mathbf{x}_0 & \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 & \mathbf{x}_2^\top \mathbf{x}_3 \\ \mathbf{x}_3^\top \mathbf{x}_0 & \mathbf{x}_3^\top \mathbf{x}_1 & \mathbf{x}_3^\top \mathbf{x}_2 & \mathbf{x}_3^\top \mathbf{x}_3 \end{bmatrix}$$

For the toy embedding tensor, the 10 unique entries are:

Diagonal entries (self-similarity, equal to the squared norm $\|\mathbf{x}_i\|^2$):

$$S_{00} = (0.1)(0.1) + (-0.4)(-0.4) + (0.2)(0.2) = \mathbf{0.21}$$

$$S_{11} = (0.5)(0.5) + (0.1)(0.1) + (-0.8)(-0.8) = \mathbf{0.90}$$

$$S_{22} = (-0.3)(-0.3) + (0.9)(0.9) + (0.4)(0.4) = \mathbf{1.06}$$

$$S_{33} = (0.2)(0.2) + (-0.2)(-0.2) + (0.1)(0.1) = \mathbf{0.09}$$

Off-diagonal entries (pairwise similarity between distinct tokens):

$$S_{01} = (0.1)(0.5) + (-0.4)(0.1) + (0.2)(-0.8) = \mathbf{-0.15}$$

$$S_{02} = (0.1)(-0.3) + (-0.4)(0.9) + (0.2)(0.4) = \mathbf{-0.31}$$

$$S_{03} = (0.1)(0.2) + (-0.4)(-0.2) + (0.2)(0.1) = \mathbf{0.12}$$

$$S_{12} = (0.5)(-0.3) + (0.1)(0.9) + (-0.8)(0.4) = \mathbf{-0.38}$$

$$S_{13} = (0.5)(0.2) + (0.1)(-0.2) + (-0.8)(0.1) = \mathbf{0.00}$$

$$S_{23} = (-0.3)(0.2) + (0.9)(-0.2) + (0.4)(0.1) = \mathbf{-0.20}$$

The complete similarity matrix:

$$S = XX^\top = \begin{bmatrix} 0.21 & -0.15 & -0.31 & 0.12 \\ -0.15 & 0.90 & -0.38 & 0.00 \\ -0.31 & -0.38 & 1.06 & -0.20 \\ 0.12 & 0.00 & -0.20 & 0.09 \end{bmatrix}$$

The matrix is symmetric ($S_{ij} = S_{ji}$) because the dot product is commutative. Each entry is a scalar that quantifies the geometric relationship between two embedding vectors. For instance, $S_{12} = -0.38$ indicates that the embeddings for `quick` and `brown` point in roughly opposite directions, while $S_{03} = 0.12$ indicates a mild positive alignment between `The` and `fox`.

The Reversal Permutation

Consider the reversed sequence `fox` `brown` `quick` `The`. The permutation matrix that maps the original ordering to this reversal is:

$$P = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Each row of P contains a single 1 that specifies which row of X is pulled into that position:

- $P_{03} = 1$: row 0 of PX receives row 3 of X (the embedding for `fox`)
- $P_{12} = 1$: row 1 of PX receives row 2 of X (the embedding for `brown`)
- $P_{21} = 1$: row 2 of PX receives row 1 of X (the embedding for `quick`)
- $P_{30} = 1$: row 3 of PX receives row 0 of X (the embedding for `The`)

The permuted embedding tensor:

$$PX = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0.1 & -0.4 & 0.2 \\ 0.5 & 0.1 & -0.8 \\ -0.3 & 0.9 & 0.4 \\ 0.2 & -0.2 & 0.1 \end{bmatrix} = \begin{bmatrix} 0.2 & -0.2 & 0.1 \\ -0.3 & 0.9 & 0.4 \\ 0.5 & 0.1 & -0.8 \\ 0.1 & -0.4 & 0.2 \end{bmatrix}$$

Row 0 of PX is $\begin{bmatrix} 0.2 & -0.2 & 0.1 \end{bmatrix}$, the embedding for `fox`. Row 1 is `brown`, row 2 is `quick`, and row 3 is `The`. The three-dimensional vectors are identical to their counterparts in X; only their row assignments have changed.

Permutation Invariance of the Similarity Matrix

The similarity matrix of the permuted tensor is:

$$S' = (PX)(PX)^\top$$

Expanding the transpose of a product using the identity $(AB)^\top = B^\top A^\top$:

$$S' = (PX)(PX)^\top = (PX)(X^\top P^\top) = P(XX^\top)P^\top = PSP^\top$$

This algebraic identity ($S' = PSP^\top$) holds for any permutation matrix P and any embedding tensor X . In linear algebra, the operation PSP^{-1} is known as matrix conjugation. For any permutation matrix, its inverse is equal to its transpose ($P^{-1} = P^\top$). This means that if P applies a permutation, $P^\top$ precisely reverses it.

Conjugating a matrix by a permutation matrix produces a highly specific structural effect. Multiplying by P on the left rearranges the rows of S , and multiplying by $P^\top$ on the right rearranges the columns of S in the exact same order.

To visualize this dual rearrangement, consider a 2×2 similarity matrix S and the 2×2 swap permutation P :

$$S = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \qquad P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = P^\top$$

Multiplying S on the left by P swaps the rows:

$$PS = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \begin{bmatrix} C & D \\ A & B \end{bmatrix}$$

Multiplying that result on the right by $P^\top$ applies the exact same swap to the columns:

$$(PS)P^\top = \begin{bmatrix} C & D \\ A & B \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} D & C \\ B & A \end{bmatrix}$$

The pairwise similarity matrix of the permuted sequence is therefore not a newly computed set of relationships. It is the original matrix S with its grid coordinates symmetrically shuffled.

The significance of this property is that conjugation does not create, destroy, or modify a single similarity score. Every scalar value in the original matrix survives intact in the new matrix. Because a Transformer's self-attention mechanism relies entirely on these dot products to determine how tokens relate to one another, it calculates the exact same semantic relationships between words regardless of the input order. This mathematically proves that computing dot products between static embeddings is a permutation-invariant operation: it extracts semantic similarity but remains completely blind to sequence position.

Numerical Verification

Computing $S' = (PX)(PX)^\top$ directly from the permuted tensor:

$$S' = \begin{bmatrix} 0.09 & -0.20 & 0.00 & 0.12 \\ -0.20 & 1.06 & -0.38 & -0.31 \\ 0.00 & -0.38 & 0.90 & -0.15 \\ 0.12 & -0.31 & -0.15 & 0.21 \end{bmatrix}$$

Every entry of S appears in S' , relocated to a new row-column position. The similarity between any two specific tokens is unchanged:

Token Pair	Position in S	Value in S	Position in S'	Value in S'
(`The`, `quick`)	S_{01}	-0.15	S'_{32}	-0.15
(`The`, `brown`)	S_{02}	-0.31	S'_{31}	-0.31
(`The`, `fox`)	S_{03}	0.12	S'_{30}	0.12
(`quick`, `brown`)	S_{12}	-0.38	S'_{21}	-0.38
(`quick`, `fox`)	S_{13}	0.00	S'_{20}	0.00
(`brown`, `fox`)	S_{23}	-0.20	S'_{10}	-0.20

The relocation follows the permutation: `The` moved from row 0 to row 3, `quick` from row 1 to row 2, `brown` from row 2 to row 1, and `fox` from row 3 to row 0. The similarity between `The` and `quick`, originally at S_{01} (rows 0 and 1), appears at S'_{32} (rows 3 and 2) because those are the new positions of `The` and `quick`. The numerical value -0.15 is unchanged.

The Consequence

The computation of the raw similarity matrix $S = XX^\top$ serves as a diagnostic tool to expose a structural limitation in the embedding tensor itself. While a full self-attention mechanism applies learned query and key projections before computing dot products, those projections operate on the input embeddings. The raw similarity matrix demonstrates the geometric baseline that self-attention inherits.

Because the embeddings are static (the vector for `brown` is identical regardless of where it appears in the sequence), the sequences `The quick brown fox` and `fox brown quick The` produce the exact same set of pairwise similarity scores. The algebraic identity $S' = PSP^\top$ guarantees this: reordering the sequence merely shuffles the coordinates of the scores without altering their values.

This invariance is not an artifact of the toy model's small dimensions. It holds for any sequence length, any embedding dimension, and any set of embedding values. It is a structural property of the representation: the embedding tensor X encodes which words are present and what their distributional signatures are, but it encodes absolutely zero information about sequence order.

For a language model, this is a fundamental deficiency. The sentence `dog bites man` and the sentence `man bites dog` contain the same three tokens. Because their underlying embedding vectors are identical, their similarity matrices are permutations of each other. A self-attention mechanism processing these vectors cannot mathematically distinguish between the two sentences, rendering it incapable of processing sequential language.

The Requirement

To resolve this deficiency, the embedding tensor must be augmented with information that explicitly depends on sequence position, not just token identity.

Mathematically, the identity $(PM)(PM)^\top = P(MM^\top)P^\top$ is an immutable rule of linear algebra. If a permutation P is applied to any arbitrary tensor M, its similarity matrix is always conjugated by P. The goal of positional encoding is to ensure that reordering the input sequence does *not* equate to simply applying P to the final tensor.

For raw static embeddings, the tensor for the reversed sequence is exactly PX. But if a position-dependent signal is injected, the vector for `brown` at position 2 is distinct from the vector for `brown` at position 1. Reordering the input words produces an entirely new set of vectors, not a rearrangement of the original set. Because the tensor for the reversed sequence is not merely PX_{pos}, its similarity matrix $S_{reversed}$ is no longer equal to $PSP^\top$. The conjugation relationship between the two sequences is broken, allowing the self-attention mechanism to compute unique relevance scores for different word orderings.

The mechanism that injects this position-dependent information is the subject of the remaining chapters. Chapter 3 explicitly verifies this claim by injecting positional data into the toy tensor and proving that $S_{reversed} \neq PSP^\top$. First, however, the next chapter enumerates the mathematical constraints such a mechanism must satisfy and derives the sinusoidal formula that served as the historical solution.

Chapter 2: Design Constraints and the Sinusoidal Formula

The preceding chapter demonstrated that the embedding tensor X is invariant under row permutations: reordering the sequence rearranges the similarity matrix without altering any similarity score. A mechanism is needed that assigns a distinct, position-dependent vector to each location in the sequence, such that incorporating these vectors into the embedding breaks the permutation symmetry.

This mechanism cannot be arbitrary. A random vector assigned to each position would break permutation invariance, but it would also inject noise that disrupts downstream computation. The positional encoding must satisfy a precise set of mathematical constraints to be useful.

The Five Constraints

A positional encoding is a function $\mathbf{p} : \mathbb{Z}_{\geq 0} \to \mathbb{R}^{d_{model}}$. The notation $\mathbb{Z}_{\geq 0}$ denotes the set of non-negative integers $\{0, 1, 2, 3, \ldots\}$: every possible position index in a sequence. The codomain $\mathbb{R}^{d_{model}}$ is the same vector space that the embedding vectors occupy (for the toy model, $\mathbb{R}^3$). The function takes a single non-negative integer t and returns a d_{model} -dimensional vector $\mathbf{p}(t)$: one encoding vector per position. For the toy sequence `The` `quick` `brown` `fox`, the position indices are $t \in \{0, 1, 2, 3\}$, and the function produces four vectors: $\mathbf{p}(0)$ for the position occupied by `The`, $\mathbf{p}(1)$ for `quick`, $\mathbf{p}(2)$ for `brown`, and $\mathbf{p}(3)$ for `fox`.

The position index t is distinct from the indices i and j used in Chapter 1 to label rows and columns of the similarity matrix $S_{ij} = \mathbf{x}_i^\top \mathbf{x}_j$. Here, t labels a single position in the sequence. Later in this chapter, i will be reused with a different meaning: the dimension pair index in the sinusoidal formula. Five constraints govern the design of this function.

Constraint 1: Uniqueness

Each position t must map to a distinct vector. If $\mathbf{p}(t_1) = \mathbf{p}(t_2)$ for $t_1 \neq t_2$, the encoding cannot distinguish those two positions, and permutation invariance between them persists. Formally:

$$t_1 \neq t_2 \implies \mathbf{p}(t_1) \neq \mathbf{p}(t_2)$$

Constraint 2: Boundedness

The values of $\mathbf{p}(t)$ must remain bounded, regardless of t . If positional encoding values grow with position, adding them to the embedding vectors would distort the magnitude of embeddings at later positions, overwhelming the semantic signal with a positional artifact.

The notation $p_k(t)$ denotes the k -th component of the vector $\mathbf{p}(t)$, where $k \in \{0, 1, \ldots, d_{model} - 1\}$ indexes the individual dimensions of the encoding vector (for the toy model, $k \in \{0, 1, 2\}$). The index k is introduced here rather than reusing i or j because i will serve as the dimension pair index in the sinusoidal formula later in this chapter, and j already indexes columns in the similarity matrix from Chapter 1. The constraint requires that every component of every positional encoding vector remains bounded:

$$|\, p_k(t) \,| \leq 1 \quad \text{for all } t, k$$

The specific bound $[-1, 1]$ is chosen to match the scale of the embedding components. The toy model's embedding matrix X contains values ranging from -0.8 to 0.9, all within this interval. In production models, layer normalization constrains embedding components to a comparable magnitude. Because the positional encoding is added element-wise to the embedding (as derived in Chapter 3), the two signals must be of the same order: a positional encoding bounded in $[-1, 1]$ contributes a signal commensurate with the semantic content. A wider bound would allow the positional signal to dominate the embedding; a narrower bound would render it negligible.

Constraint 3: Determinism

The encoding must be a fixed, deterministic function of position, not a set of learned parameters. A learned positional embedding (a trainable matrix $W_P \in \mathbb{R}^{T_{max} \times d_{model}}$) introduces $T_{max} \times d_{model}$ additional parameters and imposes a hard maximum sequence length T_{max}. Because each position's vector is an independent parameter, the mechanism cannot generalize to positions unseen during training: if a model is trained exclusively on sequences up to 1024 tokens, it possesses no vector for position 1024 during inference, and lacks a mathematical rule to generate one. A deterministic function avoids these limitations: it requires no training, imposes no maximum length, and evaluates validly for any position index t on demand.

Constraint 4: Smoothness

Nearby positions should produce nearby encoding vectors. If $\mathbf{p}(t)$ and $\mathbf{p}(t+1)$ are distant in $\mathbb{R}^{d_{model}}$, the encoding treats adjacent positions as unrelated. This discards the assumption that sequential proximity implies structural relevance: words located next to each other are structurally more related than words separated by a large distance. Formally, the Euclidean distance $\|\mathbf{p}(t) - \mathbf{p}(t+1)\|$ should be small relative to $\|\mathbf{p}(t) - \mathbf{p}(t+\Delta)\|$ for offset $\Delta > 1$.

Constraint 5: Relative Representability

The relationship between any two positions t and $t + \Delta$ should be expressible as a fixed linear transformation that depends only on the offset Δ, not on the absolute positions. This means there exists a matrix M_Δ (independent of t) such that:

$$\mathbf{p}(t + \Delta) = M_\Delta \, \mathbf{p}(t)$$

This constraint enables the model to learn relative positional relationships (the token 3 positions ahead) rather than memorizing absolute position labels (position 7). Relative relationships generalize across different positions in the sequence and across different sequence lengths.

Trigonometric Functions as the Natural Solution

The five constraints collectively point to a narrow family of functions. The requirements of boundedness ($[-1, 1]$) and smoothness (continuous, nearby outputs for nearby inputs) immediately suggest periodic functions with bounded range. The requirement of determinism excludes learned parameters. The requirement of uniqueness excludes any single periodic function (which repeats values), but a vector of periodic functions at different frequencies can produce unique combinations across all positions, much as a vector of clock hands at different speeds (seconds, minutes, hours) uniquely identifies any moment in the day.

Trigonometric functions ($\sin$ and $\cos$) satisfy the first four constraints by construction:

- **Bounded** in $[-1, 1]$ for all inputs.
- **Smooth** (infinitely differentiable).
- **Deterministic** (fixed functions with no parameters).
- **Unique** when combined at multiple frequencies (the vector of values at different frequencies produces a distinct fingerprint for each position).

The fifth constraint (relative representability) is the decisive one. The angle-addition identities for sine and cosine provide the required linear transformation. For a single frequency ω, the value at an offset position $t + \Delta$ expands algebraically into cross-terms:

$$\sin(\omega(t + \Delta)) = \sin(\omega t)\cos(\omega\Delta) + \cos(\omega t)\sin(\omega\Delta)$$

$$\cos(\omega(t + \Delta)) = \cos(\omega t)\cos(\omega\Delta) - \sin(\omega t)\sin(\omega\Delta)$$

This system of equations can be factored into a matrix multiplication applied to the vector at position t:

$$\begin{bmatrix} \sin(\omega(t + \Delta)) \\ \cos(\omega(t + \Delta)) \end{bmatrix} = \begin{bmatrix} \cos(\omega\Delta) & \sin(\omega\Delta) \\ -\sin(\omega\Delta) & \cos(\omega\Delta) \end{bmatrix} \begin{bmatrix} \sin(\omega t) \\ \cos(\omega t) \end{bmatrix}$$

The 2×2 matrix on the right depends exclusively on the offset Δ and the frequency ω, completely independent of the absolute position t. This is the standard formulation of a two-dimensional rotation matrix. Shifting the position index from t to $t + \Delta$ increases the function's argument by exactly $\omega\Delta$. Geometrically, adding an angle to a vector's current angular position is the definition of a rotation. Thus, advancing the sequence position is mathematically equivalent to rotating the vector $[\sin, \cos]^\top$ by an angle of $\omega\Delta$.

Trigonometric functions are the unique solution to this constraint. Mathematically, the only continuous functions that can translate an additive shift ($t + \Delta$) into a fixed linear transformation (a matrix multiplication independent of t) are exponential functions and sinusoids. Because real exponentials either explode to infinity or decay to zero (violating the boundedness constraint), sinusoids remain as the sole elementary function family capable of providing this translation-invariant property while maintaining a stable magnitude.

The Sinusoidal Positional Encoding Formula

Vaswani et al. (2017) defined the positional encoding as follows. For position index t and dimension pair index i:

$$PE(t, 2i) = \sin\left(\frac{t}{10000^{\,2i\,/\,d_{model}}}\right)$$

$$PE(t, 2i+1) = \cos\left(\frac{t}{10000^{\,2i\,/\,d_{model}}}\right)$$

The formula requires two independent variables to compute a specific entry in the $T \times d_{model}$ positional encoding matrix:

- $t \in \{0, 1, \ldots, T-1\}$ is the **position index** (the matrix row). It identifies where the token sits in the sequence. To encode a single position t, the formula must generate an entire d_{model}-dimensional vector.
- i is the **dimension pair index** (the matrix column pair). It sweeps across the dimensions of the vector being generated. The maximum i is determined by halving d_{model} and zero-indexing. For a production architecture with $d_{model} = 128$, the index ranges from $i = 0$ through $i = 63$. Each increment of i fills two adjacent columns: an even dimension $2i$ with a sine evaluation, and an odd dimension $2i + 1$ with a cosine evaluation.
- d_{model} is the **embedding dimension**, which dictates the total number of frequencies needed.
- 10000 is a **base constant** that determines the minimum frequency.

The argument to the trigonometric functions can be written as $t \cdot \omega_i$, where the **angular frequency** for pair i is:

$$\omega_i = \frac{1}{10000^{\,2i \,/\, d_{model}}}$$

As the column pair index i increases, the exponent $2i/d_{model}$ increases, the denominator grows, and the frequency ω_i decreases. The first dimension pair ($i = 0$) oscillates rapidly with a wavelength of $2\pi \approx 6.28$ positions. The final dimension pair oscillates extremely slowly, taking approximately $2\pi \cdot 10000 \approx 62{,}832$ positions to complete a single cycle.

This spectrum of frequencies allows the encoding to operate at multiple scales simultaneously. The rapid oscillations act as a fine-grained signal, pinpointing exact local offsets (separating adjacent positions). The slow oscillations act as a coarse-grained signal, identifying broad regional placement (distinguishing the beginning of a document from the end).

Wavelengths for the Toy Model ($d_{model} = 3$)

For $d_{model} = 3$, the individual vector components are indexed by $k \in \{0, 1, 2\}$. The formula populates these components by iterating the pair index i:

Pair $i = 0$ (dimensions 0 and 1):

$$\omega_0 = \frac{1}{10000^{\,0/3}} = \frac{1}{10000^{0}} = \frac{1}{1} = \mathbf{1}$$

The wavelength (the number of positions for one complete cycle) is:

$$\lambda_0 = \frac{2\pi}{\omega_0} = \frac{2\pi}{1} = 2\pi \approx 6.2832 \text{ positions}$$

Dimension 0 receives $\sin(t \cdot 1) = \sin(t)$, and dimension 1 receives $\cos(t \cdot 1) = \cos(t)$. This is the fastest oscillation: the sine and cosine values cycle through a full period every ≈ 6.28 positions.

Pair $i = 1$ (dimension 2 only, the odd-dimension edge case):

$$\omega_1 = \frac{1}{10000^{\,2/3}} = \frac{1}{464.1589} \approx \mathbf{0.002154}$$

The wavelength is:

$$\lambda_1 = \frac{2\pi}{\omega_1} = 2\pi \cdot 10000^{2/3} = 2\pi \cdot 464.1589 \approx 2916.40 \text{ positions}$$

Because $d_{model} = 3$ is odd, the second pair is incomplete. Dimension $2i = 2$ exists and receives $\sin(t \cdot \omega_1)$, but the corresponding cosine component would require dimension $2i + 1 = 3$. Because the vector only has three dimensions (indexed 0, 1, and 2), this cosine component has no dimension to occupy.

The sinusoidal formula allocates frequencies in sine-cosine pairs precisely because of the algebraic expansion derived in Constraint 5. To express a relative positional offset as a rotation, the mechanism must compute a linear combination of two orthogonal components. As established by the identity $\sin(\omega(t + \Delta)) = \sin(\omega t)\cos(\omega\Delta) + \cos(\omega t)\sin(\omega\Delta)$, the unshifted cosine component $\cos(\omega t)$ is strictly required to determine the shifted sine value. Because the two components are algebraically entangled, the rotation cannot be performed without evaluating both a sine and a cosine at the same frequency.

When an architecture specifies an odd embedding dimension, the dimension count cannot accommodate full pairs. The final frequency is structurally truncated to a solitary sine component. Without its paired cosine, this final dimension cannot undergo 2D rotation and thus fails the relative representability constraint. Production architectures universally avoid this structural failure by specifying even dimensions (typically multiples of 64 or 128 per attention head).

The slow rate of change in dimension 2 is determined entirely by its assigned frequency, independent of the truncation. For this dimension, the pair index is $i = 1$, which yields the constant frequency $\omega_1 \approx 0.002154$. As the formula evaluates sequence positions from row $t = 0$ to row $t = 3$, this constant frequency is multiplied by the row index to produce the argument for the sine function ($t \cdot \omega_1$). Because the frequency is so small, the resulting argument grows extremely slowly: from 0 at the first position to just $3 \times 0.002154 \approx 0.006463$ radians at the final position. For angles this small, the sine function is nearly linear ($\sin(\theta) \approx \theta$). Consequently, dimension 2 contains values very close to zero that increase steadily, acting as a coarse positional signal across the sequence.

Computing the Positional Encoding Matrix

The positional encoding matrix $PE \in \mathbb{R}^{4\times3}$ has one row per position ($t \in \{0, 1, 2, 3\}$) and one column per dimension. To prevent the reader from having to constantly cross-reference the general formula and the wavelength derivations, the specific formula for each of the three columns is instantiated here using the computed frequencies ($\omega_0 = 1$ and $\omega_1 \approx 0.002154$):

- **Dimension 0** ($i = 0$, even): $PE(t, 0) = \sin(t \cdot \omega_0) = \sin(t \cdot 1)$
- **Dimension 1** ($i = 0$, odd): $PE(t, 1) = \cos(t \cdot \omega_0) = \cos(t \cdot 1)$
- **Dimension 2** ($i = 1$, even): $PE(t, 2) = \sin(t \cdot \omega_1) = \sin(t \cdot 0.002154)$

These three column formulas are evaluated for each row index t to produce the full matrix.

Position $t = 0$

Substituting $t = 0$ into the three column formulas:

$$PE(0, 0) = \sin(0 \cdot 1) = \sin(0) = \mathbf{0.0000}$$

$$PE(0, 1) = \cos(0 \cdot 1) = \cos(0) = \mathbf{1.0000}$$

$$PE(0, 2) = \sin(0 \cdot 0.002154) = \sin(0) = \mathbf{0.0000}$$

Position 0 receives the vector $\begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$. The sine components are zero at the origin, and the cosine component is at its maximum. These are exact values, not approximations.

Position $t = 1$

Substituting $t = 1$ into the three column formulas:

$$PE(1, 0) = \sin(1 \cdot 1) = \sin(1) = \mathbf{0.8415}$$

$$PE(1, 1) = \cos(1 \cdot 1) = \cos(1) = \mathbf{0.5403}$$

$$PE(1, 2) = \sin(1 \cdot 0.002154) = \sin(0.002154) = \mathbf{0.0022}$$

The first dimension pair ($\sin(1)$, $\cos(1)$) has moved substantially from its starting point. The argument is 1 radian, approximately 57.3° of the 2π -radian cycle. The third dimension has barely moved: $\sin(0.002154) \approx \mathbf{0.002154}$, reflecting the extremely long wavelength of the second frequency.

Position $t = 2$

Substituting $t = 2$ into the three column formulas:

$$PE(2, 0) = \sin(2 \cdot 1) = \sin(2) = \mathbf{0.9093}$$

$$PE(2, 1) = \cos(2 \cdot 1) = \cos(2) = \mathbf{-0.4161}$$

$$PE(2, 2) = \sin(2 \cdot 0.002154) = \sin(0.004309) = \mathbf{0.0043}$$

At 2 radians ($\approx 114.6^\circ$), the cosine component has crossed zero and turned negative, while the sine component is near its maximum. The third dimension has increased by another increment of approximately 0.002154 , remaining close to zero.

Position $t = 3$

Substituting $t = 3$ into the three column formulas:

$$PE(3, 0) = \sin(3 \cdot 1) = \sin(3) = \mathbf{0.1411}$$

$$PE(3, 1) = \cos(3 \cdot 1) = \cos(3) = \mathbf{-0.9900}$$

$$PE(3, 2) = \sin(3 \cdot 0.002154) = \sin(0.006463) = \mathbf{0.0065}$$

At 3 radians ($\approx 171.9°$), the sine has dropped back toward zero (it will reach zero at $\pi \approx 3.14$), and the cosine is near its negative extreme. The third dimension continues its near-linear ascent.

The Complete Positional Encoding Matrix

$$PE = \begin{bmatrix} 0.0000 & 1.0000 & 0.0000 \\ 0.8415 & 0.5403 & 0.0022 \\ 0.9093 & -0.4161 & 0.0043 \\ 0.1411 & -0.9900 & 0.0065 \end{bmatrix}_{4\times3}$$

Each row is the positional encoding vector for the corresponding position. Row 0 encodes position 0 (the location of `The` in the toy sequence), row 1 encodes position 1 (`quick`), row 2 encodes position 2 (`brown`), and row 3 encodes position 3 (`fox`). The first two columns oscillate rapidly with the pattern of $\sin(t)$ and $\cos(t)$. The third column changes by approximately 0.002 per position, reflecting the slow frequency $\omega_1 \approx 0.002154$.

Verification Against the Design Constraints

Uniqueness

To verify uniqueness, let $\mathbf{p}_t$ denote the positional encoding vector for position t (row t of the PE matrix). The Euclidean distance between any two vectors is zero if and only if the vectors are mathematically identical. Therefore, demonstrating that the distance between every pair of rows is strictly positive proves that no two rows are the same.

For example, the distance between the vectors for position 0 and position 1 is computed by summing the squared differences of their components and taking the square root:

$$\|\mathbf{p}_0 - \mathbf{p}_1\| = \sqrt{(0.0000 - 0.8415)^2 + (1.0000 - 0.5403)^2 + (0.0000 - 0.0022)^2}$$

$$\|\mathbf{p}_0 - \mathbf{p}_1\| = \sqrt{0.7081 + 0.2113 + 0.0000} \approx \mathbf{0.9589}$$

Computing the distances between all six possible pairs of rows yields:

Pair (Offset)	Distance
$\lvert\mathbf{p}_0 - \mathbf{p}_1\rvert$ ($\Delta = 1$)	0.9589
$\lvert\mathbf{p}_0 - \mathbf{p}_2\rvert$ ($\Delta = 2$)	1.6829
$\lvert\mathbf{p}_0 - \mathbf{p}_3\rvert$ ($\Delta = 3$)	1.9950
$\lvert\mathbf{p}_1 - \mathbf{p}_2\rvert$ ($\Delta = 1$)	0.9589
$\lvert\mathbf{p}_1 - \mathbf{p}_3\rvert$ ($\Delta = 2$)	1.6829
$\lvert\mathbf{p}_2 - \mathbf{p}_3\rvert$ ($\Delta = 1$)	0.9589

Every distance is strictly positive, confirming uniqueness. Furthermore, the distances reveal a specific pattern: pairs separated by the same relative offset Δ share the identical distance (e.g., all adjacent positions where $\Delta = 1$ are exactly 0.9589 units apart).

This regularity is a geometric consequence of the rotation structure. Each dimension pair traces a path along a 2D circle at its assigned frequency. The complete encoding vector, composed of multiple pairs, traces a path on a high-dimensional torus (the mathematical product of multiple circles). Because shifting position by an offset Δ applies a fixed rotational angle to each circle, a shift of Δ always translates the point by a constant Euclidean distance across the surface of the torus, independent of the absolute starting position t .

Boundedness

Every entry of PE lies in $[-1, 1]$. The minimum value is -0.9900 (dimension 1, position 3), and the maximum is 1.0000 (dimension 1, position 0). This bound holds for all positions, not just $t \in \{0, 1, 2, 3\}$. Geometrically, the trigonometric functions represent the y and x coordinates of a point traversing the unit circle (a circle with a radius of exactly 1). Because the radius is 1, no coordinate can ever exceed 1 or fall below -1 . This structural property ensures that the output of $\sin(x)$ and $\cos(x)$ for any arbitrary real input x is strictly bounded by $[-1, 1]$, perfectly satisfying the constraint.

Determinism

Every entry of PE was computed from the closed-form formula $\sin(t \cdot \omega_i)$ or $\cos(t \cdot \omega_i)$ with fixed constants ($\omega_0 = 1$, $\omega_1 \approx 0.002154$). No training was involved, no data was consulted, and the values are identical on every evaluation. Position $t = 2$ always receives the vector $[\ 0.9093 \quad -0.4161 \quad 0.0043]$, regardless of the sequence content or length.

Smoothness

The Euclidean distances between adjacent positions are:

$$\|\mathbf{p}_0 - \mathbf{p}_1\| = 0.9589, \quad \|\mathbf{p}_1 - \mathbf{p}_2\| = 0.9589, \quad \|\mathbf{p}_2 - \mathbf{p}_3\| = 0.9589$$

These are smaller than the distances between non-adjacent positions (1.6829 for $\Delta = 2$, 1.9950 for $\Delta = 3$). Nearby positions produce nearby encoding vectors, and the distance increases monotonically with the offset. The equal spacing of adjacent distances follows from the constant angular step on the unit circle traced by the first dimension pair.

Relative Representability

As derived in the trigonometric motivation above, shifting position by Δ applies a fixed linear transformation (a rotation by angle $\omega_i \Delta$) to each dimension pair. The transformation depends only on Δ , not on the absolute position t . This property is inherited directly from the angle-addition identities for sine and cosine and will be verified numerically in a later chapter.

Looking Forward

The positional encoding matrix PE is now fully computed. Each of its four rows is a three-dimensional vector that uniquely identifies a position in the sequence. The next chapter adds this matrix element-wise to the embedding tensor X from Chapter 1, producing the positionally-enriched tensor $X_{pos} = X + PE$ whose similarity matrix is no longer invariant under row permutations.

Chapter 3: Element-Wise Addition and the Positionally-Enriched Tensor

The preceding chapter derived the sinusoidal positional encoding matrix $PE \in \mathbb{R}^{4\times3}$, a deterministic function that assigns a unique, bounded vector to each position in the sequence. The embedding tensor $X \in \mathbb{R}^{4\times3}$ from Chapter 1 carries the distributional signatures of the tokens `The`, `quick`, `brown`, `fox` but is invariant under row permutations. This chapter combines the two matrices, producing a tensor in which each row encodes both the identity of the token and the position it occupies.

The Addition Operation

The positionally-enriched tensor is formed by element-wise addition:

$$X_{pos} = X + PE$$

Both X and PE are 4×3 matrices. Each entry of X_{pos} is the sum of the corresponding embedding component and the corresponding positional encoding component:

$$(X_{pos})_{tc} = X_{tc} + PE_{tc}$$

for position $t \in \{0, 1, 2, 3\}$ and dimension $c \in \{0, 1, 2\}$. The operation requires no learned parameters: it is a fixed arithmetic combination of the embedding lookup (from the Embeddings series) and the sinusoidal formula (from Chapter 2).

Computing the Enriched Tensor

The two matrices to be added:

$$X = \begin{bmatrix} 0.1 & -0.4 & 0.2 \\ 0.5 & 0.1 & -0.8 \\ -0.3 & 0.9 & 0.4 \\ 0.2 & -0.2 & 0.1 \end{bmatrix}, \quad PE = \begin{bmatrix} 0.0000 & 1.0000 & 0.0000 \\ 0.8415 & 0.5403 & 0.0022 \\ 0.9093 & -0.4161 & 0.0043 \\ 0.1411 & -0.9900 & 0.0065 \end{bmatrix}$$

Position $t = 0$ (`The`)

$$(X_{pos})_{0,0} = 0.1000 + 0.0000 = \mathbf{0.1000}$$

$$(X_{pos})_{0,1} = -0.4000 + 1.0000 = \mathbf{0.6000}$$

$$(X_{pos})_{0,2} = 0.2000 + 0.0000 = \mathbf{0.2000}$$

The positional encoding at $t = 0$ is $[0 \quad 1 \quad 0]$ (the sine components are zero, the cosine is at its maximum). Dimension 1 shifts from -0.4 to 0.6, a displacement of $+1.0$. Dimensions 0 and 2 are unchanged.

Position $t = 1$ (`quick`)

$$(X_{pos})_{1,0} = 0.5000 + 0.8415 = \mathbf{1.3415}$$

$$(X_{pos})_{1,1} = 0.1000 + 0.5403 = \mathbf{0.6403}$$

$$(X_{pos})_{1,2} = -0.8000 + 0.0022 = \mathbf{-0.7978}$$

At position 1, the $\sin(1)$ and $\cos(1)$ components carry substantial magnitude. Dimension 0 nearly triples (from 0.5 to 1.3415). Dimension 2, governed by the slow frequency $\omega_1 \approx 0.002154$, shifts by less than 0.003.

Position $t = 2$ (`brown`)

$$(X_{pos})_{2,0} = -0.3000 + 0.9093 = \mathbf{0.6093}$$

$$(X_{pos})_{2,1} = 0.9000 + (-0.4161) = \mathbf{0.4839}$$

$$(X_{pos})_{2,2} = 0.4000 + 0.0043 = \mathbf{0.4043}$$

The sign of dimension 0 flips: the original embedding component is -0.3, but the positional encoding adds 0.9093 (close to the maximum of $\sin$), pulling the sum to $+0.6093$. Dimension 1 decreases because $\cos(2) \approx \mathbf{-0.4161}$ is negative at 2 radians.

Position $t = 3$ (`fox`)

$$(X_{pos})_{3,0} = 0.2000 + 0.1411 = \mathbf{0.3411}$$

$$(X_{pos})_{3,1} = -0.2000 + (-0.9900) = \mathbf{-1.1900}$$

$$(X_{pos})_{3,2} = 0.1000 + 0.0065 = \mathbf{0.1065}$$

At 3 radians ($\approx 171.9^\circ$), $\cos(3) \approx \mathbf{-0.9900}$ is near its negative extreme, pushing dimension 1 to -1.19. This is the most extreme displacement in the matrix: the embedding component (-0.2) and the positional component (-0.99) reinforce each other.

The Complete Positionally-Enriched Tensor

$$X_{pos} = X + PE = \begin{bmatrix} 0.1000 & 0.6000 & 0.2000 \\ 1.3415 & 0.6403 & -0.7978 \\ 0.6093 & 0.4839 & 0.4043 \\ 0.3411 & -1.1900 & 0.1065 \end{bmatrix}_{4\times3}$$

Each row of X_{pos} is no longer the pure distributional signature of a token. Row 2 is not the embedding for `brown` as it appears across the entire training corpus. It is the embedding for `brown` combined with the positional fingerprint of position 2 in this specific sequence.

The Similarity Matrix After Positional Encoding

In Chapter 1, the pairwise dot-product similarity matrix $S = XX^\top$ was computed for the original embedding tensor. That matrix was invariant under row permutations: reordering the sequence rearranged the entries of S without changing any similarity value. The positionally-enriched tensor X_{pos} produces a new similarity matrix:

$$S_{pos} = X_{pos}\, X_{pos}^\top = \begin{bmatrix} 0.1000 & 0.6000 & 0.2000 \\ 1.3415 & 0.6403 & -0.7978 \\ 0.6093 & 0.4839 & 0.4043 \\ 0.3411 & -1.1900 & 0.1065 \end{bmatrix} \begin{bmatrix} 0.1000 & 1.3415 & 0.6093 & 0.3411 \\ 0.6000 & 0.6403 & 0.4839 & -1.1900 \\ 0.2000 & -0.7978 & 0.4043 & 0.1065 \end{bmatrix}$$

Each entry $(S_{pos})_{ij} = \mathbf{x}_i^{pos} \cdot \mathbf{x}_j^{pos}$ is the dot product of the enriched vectors at positions i and j.

Computing the Entries

Diagonal entries (squared norms of the enriched vectors):

$$(S_{pos})_{00} = (0.1000)^2 + (0.6000)^2 + (0.2000)^2 = \mathbf{0.41}$$

$$(S_{pos})_{11} = (1.3415)^2 + (0.6403)^2 + (-0.7978)^2 = \mathbf{2.85}$$

$$(S_{pos})_{22} = (0.6093)^2 + (0.4839)^2 + (0.4043)^2 = \mathbf{0.77}$$

$$(S_{pos})_{33} = (0.3411)^2 + (-1.1900)^2 + (0.1065)^2 = \mathbf{1.54}$$

Off-diagonal entries (pairwise similarities):

$$(S_{pos})_{01} = (0.1000)(1.3415) + (0.6000)(0.6403) + (0.2000)(-0.7978)$$

$$= 0.1341 + 0.3842 + (-0.1596) = \mathbf{0.36}$$

$$(S_{pos})_{02} = (0.1000)(0.6093) + (0.6000)(0.4839) + (0.2000)(0.4043)$$

$$= 0.0609 + 0.2903 + 0.0809 = \mathbf{0.43}$$

$$(S_{pos})_{03} = (0.1000)(0.3411) + (0.6000)(-1.1900) + (0.2000)(0.1065)$$

$$= 0.0341 + (-0.7140) + 0.0213 = \mathbf{-0.66}$$

$$(S_{pos})_{12} = (1.3415)(0.6093) + (0.6403)(0.4839) + (-0.7978)(0.4043)$$

$$= 0.8174 + 0.3098 + (-0.3226) = \mathbf{0.80}$$

$$(S_{pos})_{13} = (1.3415)(0.3411) + (0.6403)(-1.1900) + (-0.7978)(0.1065)$$

$$= 0.4576 + (-0.7620) + (-0.0849) = \mathbf{-0.39}$$

$$(S_{pos})_{23} = (0.6093)(0.3411) + (0.4839)(-1.1900) + (0.4043)(0.1065)$$

$$= 0.2078 + (-0.5758) + 0.0430 = \mathbf{-0.32}$$

The Complete Similarity Matrix

$$S_{pos} = \begin{bmatrix} 0.41 & 0.36 & 0.43 & -0.66 \\ 0.36 & 2.85 & 0.80 & -0.39 \\ 0.43 & 0.80 & 0.77 & -0.32 \\ -0.66 & -0.39 & -0.32 & 1.54 \end{bmatrix}$$

Comparison with the Original Similarity Matrix

The original similarity matrix from Chapter 1:

$$S = \begin{bmatrix} 0.21 & -0.15 & -0.31 & 0.12 \\ -0.15 & 0.90 & -0.38 & 0.00 \\ -0.31 & -0.38 & 1.06 & -0.20 \\ 0.12 & 0.00 & -0.20 & 0.09 \end{bmatrix}$$

Every entry has changed. Some qualitative relationships have reversed entirely. In the original S, the similarity between `The` (position 0) and `brown` (position 2) was $S_{02} = -0.31$ (the embeddings pointed in roughly opposite directions). After positional encoding, $(S_{pos})_{02} = 0.43$: the enriched vectors are now positively aligned, because the positional components at positions 0 and 2 have shifted both vectors into a region of the space where their dot product is positive.

The positional encoding has not merely perturbed the similarity scores. It has restructured the geometry of the representation. The enriched vectors encode a superposition of two signals (semantic content and sequential position), and the resulting dot products reflect both.

Breaking Permutation Invariance

The decisive test is whether the reversed sequence `fox` `brown` `quick` `The` still produces an equivalent similarity matrix. In Chapter 1, the reversed sequence yielded $S' = PSP^\top$: the same similarity scores, relocated to new matrix positions. The question is whether the same invariance holds after positional encoding.

The reversal changes which token occupies each position. In the original ordering, `The` occupies position 0 and `fox` occupies position 3. In the reversed ordering, `fox` occupies position 0 and `The` occupies position 3. Because the positional encoding is a function of position (not of token identity), each token now receives a different positional vector than it received in the original ordering.

The reversed embedding tensor $X_{rev} = PX$ rearranges the rows of X while leaving their vector components intact:

$$X_{rev} = \begin{bmatrix} 0.2 & -0.2 & 0.1 \\ -0.3 & 0.9 & 0.4 \\ 0.5 & 0.1 & -0.8 \\ 0.1 & -0.4 & 0.2 \end{bmatrix} \begin{matrix} \leftarrow \text{fox} \\ \leftarrow \text{brown} \\ \leftarrow \text{quick} \\ \leftarrow \text{The} \end{matrix}$$

The positionally-enriched reversed tensor is formed by adding the original positional encoding matrix PE to this reversed embedding tensor:

$$X_{pos}^{rev} = X_{rev} + PE = PX + PE$$

Note that PE is the same matrix in both cases: row t of PE always encodes position t, regardless of which token occupies that position. This is the mechanism by which positional encoding breaks the symmetry.

$$X_{pos}^{rev} = \begin{bmatrix} 0.2 & -0.2 & 0.1 \\ -0.3 & 0.9 & 0.4 \\ 0.5 & 0.1 & -0.8 \\ 0.1 & -0.4 & 0.2 \end{bmatrix} + \begin{bmatrix} 0.0000 & 1.0000 & 0.0000 \\ 0.8415 & 0.5403 & 0.0022 \\ 0.9093 & -0.4161 & 0.0043 \\ 0.1411 & -0.9900 & 0.0065 \end{bmatrix} = \begin{bmatrix} 0.2000 & 0.8000 & 0.1000 \\ 0.5415 & 1.4403 & 0.4022 \\ 1.4093 & -0.3161 & -0.7957 \\ 0.2411 & -1.3900 & 0.2065 \end{bmatrix}$$

To compute the similarity matrix of the reversed, positionally-encoded sequence, the pairwise dot products between the rows of X_{pos}^{rev} are computed by multiplying it with its transpose:

$$S_{pos}^{rev} = X_{pos}^{rev} \left(X_{pos}^{rev}\right)^{\top} = \begin{bmatrix} 0.2000 & 0.8000 & 0.1000 \\ 0.5415 & 1.4403 & 0.4022 \\ 1.4093 & -0.3161 & -0.7957 \\ 0.2411 & -1.3900 & 0.2065 \end{bmatrix} \begin{bmatrix} 0.2000 & 0.5415 & 1.4093 & 0.2411 \\ 0.8000 & 1.4403 & -0.3161 & -1.3900 \\ 0.1000 & 0.4022 & -0.7957 & 0.2065 \end{bmatrix}$$

Performing these multiplications yields the new similarity scores:

$$S_{pos}^{rev} = \begin{bmatrix} 0.69 & 1.30 & -0.05 & -1.04 \\ 1.30 & 2.53 & -0.01 & -1.79 \\ -0.05 & -0.01 & 2.72 & 0.61 \\ -1.04 & -1.79 & 0.61 & 2.03 \end{bmatrix}$$

The Symmetry Is Broken

In Chapter 1, the similarity between `fox` and `brown` was -0.20 regardless of their positions in the sequence. After positional encoding, the similarity between these two tokens depends on where they appear:

Token Pair	Original S	S_{pos} (original order)	S^{rev}_{pos} (reversed order)
(`The` , `quick`)	-0.15	0.36	0.61
(`The` , `brown`)	-0.31	0.43	-1.79
(`The` , `fox`)	0.12	-0.66	-1.04
(`quick` , `brown`)	-0.38	0.80	-0.01
(`quick` , `fox`)	0.00	-0.39	-0.05
(`brown` , `fox`)	-0.20	-0.32	1.30

No pair retains the same similarity across the two orderings. The entry for (`brown` , `fox`) shifts from -0.32 in the original ordering to 1.30 in the reversed ordering, a change not only in magnitude but in sign. The sequences `The quick brown fox` and `fox brown quick The` are no longer indistinguishable to any mechanism that computes dot-product similarities between the enriched vectors.

The algebraic reason is direct. In Chapter 1, $S' = PSP^\top$ held because PX was simply a rearrangement of the same row vectors. After positional encoding, $X^{rev}_{pos} = PX + PE \neq P(X + PE) = PX_{pos}$. The positional encoding matrix PE is added based on position, not based on token identity. Permuting the tokens and then adding position-based encodings produces different vectors than adding the encodings first and then permuting.

The Meaning of a Positionally-Enriched Vector

The vector for `brown` in the original embedding tensor X is $[-0.3 \quad 0.9 \quad 0.4]$. This is the abstract distributional signature shared by every occurrence of `brown` across the training corpus: `brown` at position 0, `brown` at position 47, `brown` at position 1000 all receive the same vector.

After positional encoding, the vector for `brown` at position 2 is $[0.6093 \quad 0.4839 \quad 0.4043]$. This is not the universal distributional signature of `brown`. It is `brown` at position 2 in this specific sequence. If `brown` appeared at position 5, it would receive a different positional encoding ($\mathbf{p}_5 \neq \mathbf{p}_2$) and produce a different enriched vector. The embedding now carries two signals: what the token means (from X) and where it occurs (from PE).

The Information-Theoretic Cost of Additive Encoding

The addition $X_{pos} = X + PE$ superimposes the semantic signal and the positional signal in the same d_{model} dimensions. The three components of the enriched vector for `brown` at position 2 ($0.6093, 0.4839, 0.4043$) encode both the distributional identity of `brown` and the positional fingerprint of position 2, entangled in each coordinate.

This element-wise addition irreversibly fuses the two signals. Given only the enriched sum 0.6093 from the first component, there is no algebraic method to separate it back into its constituent parts (-0.3 and 0.9093) without providing the independent positional vector $\mathbf{p}_2$. The original embedding and the positional encoding are permanently entangled. Because the semantic meaning of the token and its sequential position must share the exact same $d_{model} = 3$ numerical coordinates, they compete for representational space. The network must expend parameter capacity in its subsequent layers (attention and feedforward transformations) to isolate and utilize these intertwined signals.

An alternative approach would concatenate the two signals rather than add them, producing vectors of dimension $2 \cdot d_{model}$. Concatenation preserves both signals without interference but doubles the dimensionality, doubling the computational cost of every subsequent matrix multiplication. In production, d_{model} is typically 4096 or larger; doubling it is prohibitively expensive. The additive approach accepts the cost of superposition to avoid the cost of doubled dimensionality.

A third approach (introduced in Chapter 4) avoids adding anything to the embedding at all, instead encoding position through a geometric operation applied inside the attention mechanism. This avoids both the superposition cost of addition and the dimensionality cost of concatenation.

Structural Limitations of Absolute Additive Positional Encoding

The sinusoidal positional encoding satisfies the five design constraints enumerated in Chapter 2 and successfully breaks permutation invariance, as verified above. Three structural properties of the additive mechanism limit its effectiveness in practice.

Single-Layer Injection

The positional encoding is added to the embedding tensor once, at the input to the first layer of the Transformer. Subsequent layers (attention, layer normalization, feedforward networks) transform the enriched vectors through nonlinear operations. With each successive transformation, the positional signal becomes increasingly entangled with the semantic signal and increasingly diluted. By the deeper layers of a model with dozens or hundreds of layers, the positional information injected at the input may have degraded substantially.

The model has no mechanism to refresh the positional signal at intermediate layers. Whatever positional information reaches the deeper layers must survive the intervening transformations intact. Empirical evidence confirms the limitation: models using additive positional encoding show degraded position-sensitivity in their deeper layers.

Absolute Position Labels

The sinusoidal encoding assigns a fixed vector to each absolute position: position 0 always receives $[0 \quad 1 \quad 0]$, position 1 always receives $[0.8415 \quad 0.5403 \quad 0.0022]$, and so on, regardless of the sequence length or context. The token at position 5 carries the label "I am at position 5," not the relationally richer signal "I am 3 positions after the token at position 2."

Chapter 2 demonstrated that the sinusoidal formula satisfies relative representability (the relationship between any two positions can be expressed as a fixed rotation by angle $\omega\Delta$). This property exists in the encoding, but the additive injection mechanism does not exploit it directly. The positional signal is added to the embedding before the linear projections that produce the query and key vectors used in attention. After projection, the positional and semantic components are linearly combined, and the clean rotational relationship between positions is obscured by the projection weights. The model must learn to extract relative positional information from the entangled representation, rather than receiving it in a form that is relative by construction.

Length Extrapolation Degradation

A model trained on sequences of maximum length T_{train} encounters the positional encoding vectors $\mathbf{p}_0$ through $\mathbf{p}_{T_{train}-1}$ during training. At inference, if a sequence of length $T > T_{train}$ is presented, the model must process positional vectors $\mathbf{p}_{T_{train}}, \ldots, \mathbf{p}_{T-1}$ that it has never seen.

The sinusoidal formula produces well-defined values at any position t (unlike learned positional embeddings, which have no representation beyond T_{max}). The values at unseen positions are valid points on the sinusoidal curves. The difficulty is that the model's learned attention patterns were calibrated to the range of positional encodings seen during training, and the statistical distribution of enriched vectors at positions beyond T_{train} differs from what the model was trained on. In practice, this distributional shift causes performance degradation on sequences longer than the training length, even though the encoding itself remains mathematically well-defined.

Looking Forward

The three limitations share a common root: the additive mechanism treats positional encoding as a preprocessing step, applied once to the input and then left to survive the subsequent computation. The position information is a static addendum to the embedding, not an active participant in the attention computation that determines which tokens attend to which.

The next chapter introduces the query-key framework that underlies the attention mechanism and formalizes the requirement that positional encoding should produce relevance scores that depend on relative position, not absolute position labels. That requirement motivates a fundamentally different approach: encoding position not by adding a vector to the embedding, but by rotating vectors inside the attention mechanism itself.

Chapter 4: The Query-Key Framework and the Relative Position Requirement

The preceding chapter demonstrated that additive sinusoidal positional encoding breaks permutation invariance in the dot-product similarity matrix $S_{pos} = X_{pos} X_{pos}^\top$. The three structural limitations identified (single-layer injection, absolute position labels, length extrapolation degradation) share a common root: the positional signal is added to the embedding once as a preprocessing step and then left to survive the subsequent computation.

This chapter introduces the specific component of the Transformer architecture where positional information has its greatest impact: the query-key dot product that determines which positions attend to which. The full attention mechanism (scoring, normalization, value aggregation) is the subject of the Transformers series. This chapter introduces only the two projections and the dot product that Rotary Position Embeddings operate on.

Query and Key Projections

The attention mechanism requires a mechanism for each position in the sequence to assess the relevance of every other position. This assessment is performed through two learned linear projections that transform each embedding vector into two distinct roles.

Given the embedding vector $\mathbf{x}_t \in \mathbb{R}^{d_{model}}$ at position t , two projection matrices produce:

$$\mathbf{q}_t = W_Q \, \mathbf{x}_t \qquad (\text{query vector})$$

$$\mathbf{k}_t = W_K \, \mathbf{x}_t \qquad (\text{key vector})$$

where $W_Q, W_K \in \mathbb{R}^{d_{model} \times d_{model}}$ are learned parameter matrices. The query vector $\mathbf{q}_t$ encodes the question "what information does position t seek?" and the key vector $\mathbf{k}_t$ encodes the answer "what information does position t offer."

The **relevance score** between position m (the querying position) and position n (the attended position) is the dot product:

$$a_{mn} = \mathbf{q}_m^\top \mathbf{k}_n$$

Because both $\mathbf{q}_m$ and $\mathbf{k}_n$ are $d_{model} \times 1$ column vectors, the first vector must be transposed into a $1 \times d_{model}$ row vector to compute the scalar dot product via matrix multiplication. Moving the transpose to the second vector ($\mathbf{q}_m \mathbf{k}_n^\top$) would incorrectly compute a full $d_{model} \times d_{model}$ matrix. Reversing the order of the vectors entirely ($\mathbf{k}_n^\top \mathbf{q}_m$) computes the exact same scalar value, but standard mathematical convention places the querying vector on the left.

A large positive score indicates that position m considers position n highly relevant; a score near zero indicates low relevance; a negative score indicates that the content at position n is actively irrelevant to the query at position m . The matrix $A \in \mathbb{R}^{T \times T}$ of all pairwise relevance scores determines the flow of information across the sequence.

The score a_{mn} is not symmetric in general: position m may consider position n relevant without the converse holding, because $W_Q \neq W_K$ and therefore $\mathbf{q}_m^\top \mathbf{k}_n \neq \mathbf{q}_n^\top \mathbf{k}_m$.

Defining the Projection Matrices

For the toy model with $d_{model} = 3$, the projection matrices W_Q and W_K are 3×3 . The following hand-chosen values produce non-trivial projections with entries small enough for manual verification:

$$W_Q = \begin{bmatrix} 0.2 & 0.1 & -0.3 \\ -0.1 & 0.4 & 0.2 \\ 0.3 & -0.2 & 0.5 \end{bmatrix}, \qquad W_K = \begin{bmatrix} 0.1 & 0.3 & -0.2 \\ 0.4 & -0.1 & 0.1 \\ -0.2 & 0.2 & 0.3 \end{bmatrix}$$

In a trained model, these matrices are learned through gradient descent. The distinct values of W_Q and W_K allow query and key vectors to capture different aspects of the same embedding vector, which is essential for the asymmetric nature of the relevance computation.

Computing the Query Vectors

Each query vector $\mathbf{q}_t = W_Q \, \mathbf{x}_t$ is a matrix-vector product. For the four positions of the toy sequence:

Position $t = 0$ (`The`), $\mathbf{x}_0 = \begin{bmatrix} 0.1 & -0.4 & 0.2 \end{bmatrix}^\top$

$q_0[0] = (0.2)(0.1) + (0.1)(-0.4) + (-0.3)(0.2) = \mathbf{-0.08}$

$q_0[1] = (-0.1)(0.1) + (0.4)(-0.4) + (0.2)(0.2) = \mathbf{-0.13}$

$q_0[2] = (0.3)(0.1) + (-0.2)(-0.4) + (0.5)(0.2) = \mathbf{0.21}$

$$\mathbf{q}_0 = \begin{bmatrix} -0.08 \\ -0.13 \\ 0.21 \end{bmatrix}$$

Position $t = 1$ (`quick`), $\mathbf{x}_1 = \begin{bmatrix} 0.5 & 0.1 & -0.8 \end{bmatrix}^\top$

$q_1[0] = (0.2)(0.5) + (0.1)(0.1) + (-0.3)(-0.8) = \mathbf{0.35}$

$q_1[1] = (-0.1)(0.5) + (0.4)(0.1) + (0.2)(-0.8) = \mathbf{-0.17}$

$q_1[2] = (0.3)(0.5) + (-0.2)(0.1) + (0.5)(-0.8) = \mathbf{-0.27}$

$$\mathbf{q}_1 = \begin{bmatrix} 0.35 \\ -0.17 \\ -0.27 \end{bmatrix}$$

Position $t = 2$ (`brown`), $\mathbf{x}_2 = [-0.3 \quad 0.9 \quad 0.4]^\top$

$q_2[0] = (0.2)(-0.3) + (0.1)(0.9) + (-0.3)(0.4) = \mathbf{-0.09}$

$q_2[1] = (-0.1)(-0.3) + (0.4)(0.9) + (0.2)(0.4) = \mathbf{0.47}$

$q_2[2] = (0.3)(-0.3) + (-0.2)(0.9) + (0.5)(0.4) = \mathbf{-0.07}$

$$\mathbf{q}_2 = \begin{bmatrix} -0.09 \\ 0.47 \\ -0.07 \end{bmatrix}$$

Position $t = 3$ (`fox`), $\mathbf{x}_3 = [\ 0.2 \quad -0.2 \quad 0.1]^\top$

$q_3[0] = (0.2)(0.2) + (0.1)(-0.2) + (-0.3)(0.1) = \mathbf{-0.01}$

$q_3[1] = (-0.1)(0.2) + (0.4)(-0.2) + (0.2)(0.1) = \mathbf{-0.08}$

$q_3[2] = (0.3)(0.2) + (-0.2)(-0.2) + (0.5)(0.1) = \mathbf{0.15}$

$$\mathbf{q}_3 = \begin{bmatrix} -0.01 \\ -0.08 \\ 0.15 \end{bmatrix}$$

Computing the Key Vectors

Each key vector $\mathbf{k}_t = W_K \mathbf{x}_t$ is computed identically, using W_K in place of W_Q.

Position $t = 0$ (`The`)

$k_0[0] = (0.1)(0.1) + (0.3)(-0.4) + (-0.2)(0.2) = \mathbf{-0.15}$

$k_0[1] = (0.4)(0.1) + (-0.1)(-0.4) + (0.1)(0.2) = \mathbf{0.10}$

$k_0[2] = (-0.2)(0.1) + (0.2)(-0.4) + (0.3)(0.2) = \mathbf{-0.04}$

$$\mathbf{k}_0 = \begin{bmatrix} -0.15 \\ 0.10 \\ -0.04 \end{bmatrix}$$

Position $t = 1$ (`quick`)

$k_1[0] = (0.1)(0.5) + (0.3)(0.1) + (-0.2)(-0.8) = \mathbf{0.24}$

$$k_1[1] = (0.4)(0.5) + (-0.1)(0.1) + (0.1)(-0.8) = \mathbf{0.11}$$

$$k_1[2] = (-0.2)(0.5) + (0.2)(0.1) + (0.3)(-0.8) = \mathbf{-0.32}$$

$$\mathbf{k}_1 = \begin{bmatrix} 0.24 \\ 0.11 \\ -0.32 \end{bmatrix}$$

Position $t = 2$ (`brown`)

$$k_2[0] = (0.1)(-0.3) + (0.3)(0.9) + (-0.2)(0.4) = \mathbf{0.16}$$

$$k_2[1] = (0.4)(-0.3) + (-0.1)(0.9) + (0.1)(0.4) = \mathbf{-0.17}$$

$$k_2[2] = (-0.2)(-0.3) + (0.2)(0.9) + (0.3)(0.4) = \mathbf{0.36}$$

$$\mathbf{k}_2 = \begin{bmatrix} 0.16 \\ -0.17 \\ 0.36 \end{bmatrix}$$

Position $t = 3$ (`fox`)

$$k_3[0] = (0.1)(0.2) + (0.3)(-0.2) + (-0.2)(0.1) = \mathbf{-0.06}$$

$$k_3[1] = (0.4)(0.2) + (-0.1)(-0.2) + (0.1)(0.1) = \mathbf{0.11}$$

$$k_3[2] = (-0.2)(0.2) + (0.2)(-0.2) + (0.3)(0.1) = \mathbf{-0.05}$$

$$\mathbf{k}_3 = \begin{bmatrix} -0.06 \\ 0.11 \\ -0.05 \end{bmatrix}$$

The Relevance Score Matrix

The full 4×4 relevance score matrix A contains the dot product $a_{ij} = \mathbf{q}_i^\top \mathbf{k}_j$ for every pair of positions. Each entry measures how relevant position j is to the query at position i.

Row 0 (`The` queries each position):

$$a_{00} = (-0.08)(-0.15) + (-0.13)(0.10) + (0.21)(-0.04) = \mathbf{-0.0094}$$

$$a_{01} = (-0.08)(0.24) + (-0.13)(0.11) + (0.21)(-0.32) = \mathbf{-0.1007}$$

$$a_{02} = (-0.08)(0.16) + (-0.13)(-0.17) + (0.21)(0.36) = \mathbf{0.0849}$$

$$a_{03} = (-0.08)(-0.06) + (-0.13)(0.11) + (0.21)(-0.05) = \mathbf{-0.0200}$$

Row 1 (`quick` queries each position):

$$a_{10} = (0.35)(-0.15) + (-0.17)(0.10) + (-0.27)(-0.04) = \mathbf{-0.0587}$$

$$a_{11} = (0.35)(0.24) + (-0.17)(0.11) + (-0.27)(-0.32) = \mathbf{0.1517}$$

$$a_{12} = (0.35)(0.16) + (-0.17)(-0.17) + (-0.27)(0.36) = \mathbf{-0.0123}$$

$$a_{13} = (0.35)(-0.06) + (-0.17)(0.11) + (-0.27)(-0.05) = \mathbf{-0.0262}$$

Row 2 (`brown` queries each position):

$$a_{20} = (-0.09)(-0.15) + (0.47)(0.10) + (-0.07)(-0.04) = \mathbf{0.0633}$$

$$a_{21} = (-0.09)(0.24) + (0.47)(0.11) + (-0.07)(-0.32) = \mathbf{0.0525}$$

$$a_{22} = (-0.09)(0.16) + (0.47)(-0.17) + (-0.07)(0.36) = \mathbf{-0.1195}$$

$$a_{23} = (-0.09)(-0.06) + (0.47)(0.11) + (-0.07)(-0.05) = \mathbf{0.0606}$$

Row 3 (`fox` queries each position):

$$a_{30} = (-0.01)(-0.15) + (-0.08)(0.10) + (0.15)(-0.04) = \mathbf{-0.0125}$$

$$a_{31} = (-0.01)(0.24) + (-0.08)(0.11) + (0.15)(-0.32) = \mathbf{-0.0592}$$

$$a_{32} = (-0.01)(0.16) + (-0.08)(-0.17) + (0.15)(0.36) = \mathbf{0.0660}$$

$$a_{33} = (-0.01)(-0.06) + (-0.08)(0.11) + (0.15)(-0.05) = \mathbf{-0.0157}$$

The Complete Relevance Score Matrix

$$A = \begin{bmatrix} -0.0094 & -0.1007 & 0.0849 & -0.0200 \\ -0.0587 & 0.1517 & -0.0123 & -0.0262 \\ 0.0633 & 0.0525 & -0.1195 & 0.0606 \\ -0.0125 & -0.0592 & 0.0660 & -0.0157 \end{bmatrix}$$

The matrix is not symmetric: $a_{01} = -0.1007$ but $a_{10} = -0.0587$. Position 0 (`The`) considers position 1 (`quick`) moderately irrelevant, while position 1 (`quick`) considers position 0 (`The`) less irrelevant. This asymmetry is a direct consequence of $W_Q \neq W_K$: the two matrices extract different linear combinations of the embedding components, so the dot product $\mathbf{q}_m^\top \mathbf{k}_n$ is not the same operation as $\mathbf{q}_n^\top \mathbf{k}_m$.

Permutation Invariance in the Relevance Score Matrix

The similarity matrix $S = XX^\top$ from Chapter 1 exhibited permutation invariance: reversing the sequence produced $S' = PSP^\top$, a rearrangement of the same scores. The relevance score matrix A inherits this invariance.

The step-by-step vector projections ($\mathbf{q}_t = W_Q\mathbf{x}_t$) treat each embedding as a 3×1 column vector. In the full 4×3 embedding tensor X, however, each token's embedding is a row ($\mathbf{x}_t^\top$).

To compute the entire set of queries simultaneously as a single matrix operation, the projection equation is transposed:

$$\mathbf{q}_t^\top = (W_Q\mathbf{x}_t)^\top = \mathbf{x}_t^\top W_Q^\top$$

Multiplying a row vector by $W_Q^\top$ projects it into a query row. Stacking all four positions into the full 4×3 matrix Q reveals the complete operation:

$$Q = \begin{bmatrix} \leftarrow & \mathbf{q}_0^\top & \rightarrow \\ \leftarrow & \mathbf{q}_1^\top & \rightarrow \\ \leftarrow & \mathbf{q}_2^\top & \rightarrow \\ \leftarrow & \mathbf{q}_3^\top & \rightarrow \end{bmatrix} = \begin{bmatrix} \leftarrow & \mathbf{x}_0^\top & \rightarrow \\ \leftarrow & \mathbf{x}_1^\top & \rightarrow \\ \leftarrow & \mathbf{x}_2^\top & \rightarrow \\ \leftarrow & \mathbf{x}_3^\top & \rightarrow \end{bmatrix} W_Q^\top = XW_Q^\top$$

The same derivation applies to the key vectors, yielding the compact matrix definitions:

$$Q = XW_Q^\top, \qquad K = XW_K^\top$$

For the reversed sequence `fox` `brown` `quick` `The`, the embedding tensor is permuted: $X_{rev} = PX$. To see exactly why the linear projections cannot break this permutation, the row operations can be tracked visually.

First, consider projecting the permuted tensor ($X_{rev}W_Q^\top$). The permutation matrix P moves the rows of X into reverse order, and then the projection $W_Q^\top$ transforms each row:

$$Q_{rev} = (PX)W_Q^\top = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \leftarrow & \mathbf{x}_0^\top & \rightarrow \\ \leftarrow & \mathbf{x}_1^\top & \rightarrow \\ \leftarrow & \mathbf{x}_2^\top & \rightarrow \\ \leftarrow & \mathbf{x}_3^\top & \rightarrow \end{bmatrix} W_Q^\top$$

$$= \begin{bmatrix} \leftarrow & \mathbf{x}_3^\top & \rightarrow \\ \leftarrow & \mathbf{x}_2^\top & \rightarrow \\ \leftarrow & \mathbf{x}_1^\top & \rightarrow \\ \leftarrow & \mathbf{x}_0^\top & \rightarrow \end{bmatrix} W_Q^\top = \begin{bmatrix} \leftarrow & \mathbf{q}_3^\top & \rightarrow \\ \leftarrow & \mathbf{q}_2^\top & \rightarrow \\ \leftarrow & \mathbf{q}_1^\top & \rightarrow \\ \leftarrow & \mathbf{q}_0^\top & \rightarrow \end{bmatrix}$$

Now consider permuting the already-projected query matrix (PQ). Here, X is projected into query vectors first, and then the permutation matrix P shuffles those query vectors into reverse order:

$$PQ = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \left(\begin{bmatrix} \leftarrow & \mathbf{x}_0^\top & \rightarrow \\ \leftarrow & \mathbf{x}_1^\top & \rightarrow \\ \leftarrow & \mathbf{x}_2^\top & \rightarrow \\ \leftarrow & \mathbf{x}_3^\top & \rightarrow \end{bmatrix} W_Q^\top \right)$$

$$= \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \leftarrow & \mathbf{q}_0^\top & \rightarrow \\ \leftarrow & \mathbf{q}_1^\top & \rightarrow \\ \leftarrow & \mathbf{q}_2^\top & \rightarrow \\ \leftarrow & \mathbf{q}_3^\top & \rightarrow \end{bmatrix} = \begin{bmatrix} \leftarrow & \mathbf{q}_3^\top & \rightarrow \\ \leftarrow & \mathbf{q}_2^\top & \rightarrow \\ \leftarrow & \mathbf{q}_1^\top & \rightarrow \\ \leftarrow & \mathbf{q}_0^\top & \rightarrow \end{bmatrix}$$

Both paths produce the exact same matrix. Multiplication on the left (PX) rearranges the row order without modifying the vectors themselves. Multiplication on the right ($XW_Q^\top$) projects each vector independently; the query vector for a given token is computed solely from its own embedding, completely unaffected by the other tokens in the sequence. Because the left-side permutation only alters positions and the right-side projection only alters internal values, the two operations never interfere. Projecting a shuffled sequence yields the exact same matrix as shuffling a projected sequence.

The same visual expansion holds for the key matrix ($K_{rev} = PK$).

This property is significant because it proves that the linear projections of the attention mechanism cannot break permutation invariance on their own. The relevance score matrix for the permuted sequence inherits the exact same invariance as the original dot-product similarity matrix $S = XX^\top$:

$$A_{rev} = Q_{rev} K_{rev}^\top = (PQ)(PK)^\top = PQK^\top P^\top = PAP^\top$$

The same algebraic identity that governed $S' = PSP^\top$ in Chapter 1 governs the relevance score matrix. Computing A_{rev} directly from the permuted embedding tensor:

$$A_{rev} = \begin{bmatrix} -0.0157 & 0.0660 & -0.0592 & -0.0125 \\ 0.0606 & -0.1195 & 0.0525 & 0.0633 \\ -0.0262 & -0.0123 & 0.1517 & -0.0587 \\ -0.0200 & 0.0849 & -0.1007 & -0.0094 \end{bmatrix}$$

Every score in A appears in A_{rev} , relocated according to the permutation. The self-relevance of `quick` is $a_{11} = 0.1517$ in the original ordering and $(A_{rev})_{22} = 0.1517$ in the reversed ordering (because `quick` moved from position 1 to position 2). The score from `brown` querying `fox` is $a_{23} = 0.0606$ in the original and $(A_{rev})_{10} = 0.0606$ in the reversed ordering.

Token Pair	Score in A	Position in A	Score in A_{rev}	Position in A_{rev}
`The` queries `quick`	-0.1007	A_{01}	-0.1007	$(A_{rev})_{32}$
`quick` queries `quick`	0.1517	A_{11}	0.1517	$(A_{rev})_{22}$
`quick` queries `brown`	-0.0123	A_{12}	-0.0123	$(A_{rev})_{21}$
`brown` queries `fox`	0.0606	A_{23}	0.0606	$(A_{rev})_{10}$
`fox` queries `brown`	0.0660	A_{32}	0.0660	$(A_{rev})_{01}$

The permutation invariance problem from Chapter 1 manifests identically in the query-key relevance scores. The sequences `The` `quick` `brown` `fox` and `fox` `brown` `quick` `The` produce the same set of relevance scores, differing only in their assignment to matrix positions. No mechanism operating on these scores can distinguish the two orderings.

The Relative Position Requirement

The fundamental problem is not merely that the relevance scores lack positional information. The problem is more specific: the ideal positional encoding should produce scores that depend on the *relative* offset between positions, not on their absolute indices.

The requirement can be stated formally. A positional encoding function f transforms query and key vectors into position-aware variants:

$$\tilde{\mathbf{q}}_m = f(\mathbf{q}_m, m), \qquad \tilde{\mathbf{k}}_n = f(\mathbf{k}_n, n)$$

The encoded relevance score is:

$$\tilde{a}_{mn} = \tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n$$

The **relative position requirement** is that $\tilde{a}_{mn}$ depends on the content vectors $\mathbf{q}_m, \mathbf{k}_n$ and the relative offset $\Delta = m - n$, but not on the absolute positions m and n individually. Formally, for any positions m, n, m', n' with $m - n = m' - n'$:

$$f(\mathbf{q}, m)^\top f(\mathbf{k}, n) = f(\mathbf{q}, m')^\top f(\mathbf{k}, n')$$

for the same content vectors $\mathbf{q}$ and $\mathbf{k}$. The score between `quick` and `fox` separated by an offset of 2 should be the same whether they occupy positions $(1, 3)$ or positions $(5, 7)$ or positions $(100, 102)$, provided the content vectors are identical.

This requirement aligns with a linguistic observation: the relationship between two tokens typically depends on how far apart they are, not on where in the sequence they happen to appear. A verb that is three positions after its subject carries the same syntactic relationship regardless of whether the subject appears at position 0 or position 50.

Why Additive Sinusoidal PE Fails the Relative Position Requirement

Chapter 3 demonstrated that additive sinusoidal PE successfully breaks permutation invariance. The question is whether it satisfies the relative position requirement in the query-key dot product.

With additive PE, the positionally-enriched embedding is $\mathbf{x}_t^{pos} = \mathbf{x}_t + \mathbf{p}_t$, where $\mathbf{p}_t$ is the sinusoidal encoding at position t. The query and key vectors become:

$$\mathbf{q}_m^{pos} = W_Q(\mathbf{x}_m + \mathbf{p}_m) = W_Q\mathbf{x}_m + W_Q\mathbf{p}_m = \mathbf{q}_m + W_Q\mathbf{p}_m$$

$$\mathbf{k}_n^{pos} = W_K(\mathbf{x}_n + \mathbf{p}_n) = W_K\mathbf{x}_n + W_K\mathbf{p}_n = \mathbf{k}_n + W_K\mathbf{p}_n$$

The linearity of the projection produces a clean decomposition: each positionally-encoded query (or key) is the sum of a content component and a position component. The relevance score expands as a four-term product:

$$(\mathbf{q}_m^{pos})^\top \mathbf{k}_n^{pos} = (\mathbf{q}_m + W_Q\mathbf{p}_m)^\top(\mathbf{k}_n + W_K\mathbf{p}_n)$$

$$= \underbrace{\mathbf{q}_m^\top \mathbf{k}_n}_{\text{content-content}} + \underbrace{\mathbf{q}_m^\top W_K\mathbf{p}_n}_{\text{content-position}} + \underbrace{(W_Q\mathbf{p}_m)^\top \mathbf{k}_n}_{\text{position-content}} + \underbrace{(W_Q\mathbf{p}_m)^\top W_K\mathbf{p}_n}_{\text{position-position}}$$

The first term is the original content-only relevance score (invariant to position). The fourth term depends only on the positions m and n (invariant to content). But the second and third terms are cross-terms that entangle content and absolute position: $\mathbf{q}_m^\top W_K\mathbf{p}_n$ depends on the content at position m and the absolute position n, not the relative offset $m - n$.

To verify concretely, the four-term decomposition for the pair (`The`, `quick`) at positions $(m = 0, n = 1)$:

$$\text{content-content: } \mathbf{q}_0^\top \mathbf{k}_1 = -0.1007$$

$$\text{content-position: } \mathbf{q}_0^\top W_K\mathbf{p}_1 = -0.0689$$

$$\text{position-content: } (W_Q\mathbf{p}_0)^\top \mathbf{k}_1 = 0.1320$$

$$\text{position-position: } (W_Q\mathbf{p}_0)^\top W_K\mathbf{p}_1 = 0.1496$$

$$\text{sum} = -0.1007 + (-0.0689) + 0.1320 + 0.1496 = \mathbf{0.1120}$$

The cross-terms (-0.0689 and 0.1320) depend on the specific absolute positions 0 and 1, not on the offset $\Delta = 0 - 1 = -1$. If the same tokens appeared at positions $(5, 6)$ (same offset $\Delta = -1$, same content), the cross-terms would differ because $\mathbf{p}_5 \neq \mathbf{p}_0$ and $\mathbf{p}_6 \neq \mathbf{p}_1$.

The sinusoidal formula does encode a rotational relationship between positions: Chapter 2 demonstrated that the positional encoding satisfies relative representability (the relationship between any two positions can be expressed as a fixed rotation). But the additive injection mechanism does not preserve this relationship through the projection. After multiplication by W_Q and W_K, the rotational structure of the positional encodings is obscured by the projection weights, and the clean relative-distance property is lost.

Preview: Rotation as Position Encoding

The relative position requirement demands that the relevance score $\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n$ depend only on content and relative offset. The failure of additive PE stems from a structural mismatch: the positional signal is injected *before* the projection, and the projection entangles it with content in a way that breaks the relative-distance property.

The solution is to encode position *after* the projection, directly on the query and key vectors themselves. Rotary Position Embeddings (RoPE) apply a position-dependent rotation to each query and key vector:

$$\tilde{\mathbf{q}}_m = R_m \, \mathbf{q}_m, \qquad \tilde{\mathbf{k}}_n = R_n \, \mathbf{k}_n$$

where R_t is a rotation matrix that depends on position t . The dot product of rotated vectors satisfies:

$$\tilde{\mathbf{q}}_m^\top \tilde{\mathbf{k}}_n = (R_m \mathbf{q}_m)^\top (R_n \mathbf{k}_n) = \mathbf{q}_m^\top R_m^\top R_n \, \mathbf{k}_n = \mathbf{q}_m^\top R_{n-m} \, \mathbf{k}_n$$

The product $R_m^\top R_n$ collapses to a single rotation R_{n-m} that depends only on the offset $\Delta = n - m$, because rotation matrices compose by adding angles. The resulting score depends on the content vectors and the relative offset, satisfying the requirement by algebraic construction rather than by learned approximation.

Nothing is added to the embedding vectors. No information is superimposed in the same dimensions. The positional signal is encoded in the geometric orientation of the query and key vectors, not in their coordinates. The next chapter derives the rotation matrix, computes it for every position in the toy model, and applies it to the query and key vectors computed in this chapter.

Chapter 5: Rotary Position Embeddings: Deriving the Rotation Matrix

The preceding chapter identified the structural failure of additive positional encoding in the query-key dot product: the cross-terms between content and position depend on absolute position indices, not on relative offsets. The preview at the end of that chapter stated that Rotary Position Embeddings (RoPE) encode position by rotating query and key vectors after projection, and that the dot product of rotated vectors depends only on relative position by construction.

This chapter derives the rotation matrix that performs this encoding. The derivation begins with complex number multiplication (the conceptual foundation), recovers the real-valued rotation matrix (the computational mechanism), and applies it to every query and key vector from the toy model. The proof that the dot product depends only on relative position is the subject of the next chapter.

The RoPE Operation

The RoPE operation (Su et al., 2021) transforms any given vector $\mathbf{v} \in \mathbb{R}^{d_{model}}$ at position t (which will serve as the query $\mathbf{q}_t$ or key $\mathbf{k}_t$) by applying a position-dependent rotation:

$$f(\mathbf{v}, t) = R_t \mathbf{v}$$

The matrix R_t is block-diagonal. Each pair of dimensions $(2i, 2i+1)$ receives a 2×2 rotation block:

$$R_t^{(i)} = \begin{bmatrix} \cos(t \cdot \theta_i) & -\sin(t \cdot \theta_i) \\ \sin(t \cdot \theta_i) & \cos(t \cdot \theta_i) \end{bmatrix}$$

The angle of rotation is $t \cdot \theta_i$, where θ_i is the angular frequency for dimension pair i :

$$\theta_i = \frac{1}{10000^{\,2i \,/\, d_{model}}}$$

The variable θ_i represents the angular frequency. Although ω is the standard physics notation for frequency, θ_i is the canonical notation established by Su et al. (2021). The value θ_i specifies the rate of rotation (radians per position step), and $t \cdot \theta_i$ yields the total angle of rotation at position t .

This frequency base is identical to the one used in sinusoidal positional encoding (Chapter 2). The motivation for rotational encoding stems directly from the relative position requirement established in Chapter 4. The dot product between a query vector and a key vector is determined by the angle between them. If a query at position m is rotated by an angle proportional to m , and a key at position n is rotated by an angle proportional to n , the angle between the two vectors changes by an amount proportional to $n - m$. The dot product becomes a function of relative distance by pure geometric construction.

Deriving the Rotation Matrix from Complex Multiplication

Rotating a vector in a 2D real plane requires matrix multiplication. Mapping the 2D plane to the complex plane provides a mathematical shortcut, collapsing 2D matrix rotation into a single scalar multiplication. Complex number arithmetic serves as an algebraic bridge to derive the real 2×2 rotation matrix without geometric proofs. The derivation has four steps.

Step 1: Represent a Dimension Pair as a Complex Number

The index i here identifies the *dimension pair* (e.g., $i = 0$ is the first pair, $i = 1$ is the second pair). For a given pair i , the two real components v_{2i} and v_{2i+1} of the generic vector $\mathbf{v}$ are treated as the real and imaginary parts of a complex number:

$$z = v_{2i} + j \cdot v_{2i+1}$$

where j is the imaginary unit ($j^2 = -1$). This maps a pair of real dimensions to a single point in the complex plane: v_{2i} determines the horizontal coordinate, v_{2i+1} determines the vertical coordinate.

Step 2: Multiply by the Position-Dependent Phase Factor

Euler's formula defines a point on the unit circle at angle α :

$$e^{j\alpha} = \cos(\alpha) + j\sin(\alpha)$$

Multiplying any complex number z by a point on the unit circle rotates z by that angle without altering its magnitude. To encode position t , the angle is set to $\alpha = t \cdot \theta_i$. The complex number z is multiplied by the unit complex exponential $e^{j\,t\,\theta_i}$:

$$e^{j\,t\,\theta_i} = \cos(t\,\theta_i) + j\,\sin(t\,\theta_i)$$

By defining the rotation angle as a multiple of t , the absolute position index is translated directly into a geometric orientation.

The rotated complex number is:

$$z' = z \cdot e^{j\,t\,\theta_i} = (v_{2i} + j\,v_{2i+1})(\cos(t\theta_i) + j\,\sin(t\theta_i))$$

Step 3: Expand the Complex Product

Expanding the complex product using the distributive property and $j^2 = -1$:

$$z' = v_{2i}\cos(t\theta_i) + j\,v_{2i}\sin(t\theta_i) + j\,v_{2i+1}\cos(t\theta_i) + j^2\,v_{2i+1}\sin(t\theta_i)$$

$$= v_{2i}\cos(t\theta_i) + j\,v_{2i}\sin(t\theta_i) + j\,v_{2i+1}\cos(t\theta_i) - v_{2i+1}\sin(t\theta_i)$$

Collecting the real terms (those without j) and the imaginary terms (those with j):

$$\text{Re}(z') = v_{2i}\cos(t\theta_i) - v_{2i+1}\sin(t\theta_i)$$

$$\text{Im}(z') = v_{2i}\sin(t\theta_i) + v_{2i+1}\cos(t\theta_i)$$

Step 4: Map Back to the Real Vector Space

Transformers do not process complex numbers; they operate exclusively on real-valued tensors. The complex plane was merely a temporary computational workspace used to perform the rotation easily. Now that the rotation is complete, the new complex number z' must be disassembled back into two standard real numbers.

The original, unrotated inputs are v_{2i} and v_{2i+1}. The goal is to compute their rotated outputs, which are denoted with primes (v'_{2i} and v'_{2i+1}) to distinguish them from the inputs.

Because Step 1 mapped dimension $2i$ to the real part and dimension $2i+1$ to the imaginary part, the exact reverse mapping is applied to z' to extract the final outputs. The new real part becomes the rotated output v'_{2i}, and the new imaginary coefficient becomes the rotated output v'_{2i+1}:

$$v'_{2i} = \text{Re}(z') = v_{2i}\cos(t\theta_i) - v_{2i+1}\sin(t\theta_i)$$

$$v'_{2i+1} = \text{Im}(z') = v_{2i}\sin(t\theta_i) + v_{2i+1}\cos(t\theta_i)$$

To convert this system of two linear equations into a single matrix multiplication, the input variables (v_{2i} and v_{2i+1}) must be factored out from their coefficients. Rewriting the equations to explicitly group the coefficients for each variable makes this extraction clear:

$$v'_{2i} = \big(\cos(t\theta_i)\big)v_{2i} + \big(-\sin(t\theta_i)\big)v_{2i+1}$$

$$v'_{2i+1} = \big(\sin(t\theta_i)\big)v_{2i} + \big(\cos(t\theta_i)\big)v_{2i+1}$$

By the definition of matrix-vector multiplication, these grouped coefficients become the rows of a 2×2 matrix, and the isolated variables form a column vector on the right. This factorization yields the final real-valued rotation matrix:

$$\begin{bmatrix} v'_{2i} \\ v'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{bmatrix} \begin{bmatrix} v_{2i} \\ v_{2i+1} \end{bmatrix}$$

This is the standard 2×2 rotation matrix, derived entirely from the algebraic properties of complex multiplication. The rotation angle $t \cdot \theta_i$ increases linearly with position t, so each successive position rotates the dimension pair by an additional θ_i radians.

Scaling to Full Dimension (The Block-Diagonal Matrix)

While the derivation focused on a single pair i, the complete RoPE operation applies this rotation to every pair in the vector simultaneously. For a vector with an even $d_{model} = 6$ (three dimension pairs), the full mathematical system is simply the 2×2 system repeated three times, using a different frequency θ_i for each pair:

$$v'_0 = v_0 \cos(t\theta_0) - v_1 \sin(t\theta_0)$$

$$v'_1 = v_0 \sin(t\theta_0) + v_1 \cos(t\theta_0)$$

$$v'_2 = v_2 \cos(t\theta_1) - v_3 \sin(t\theta_1)$$

$$v'_3 = v_2 \sin(t\theta_1) + v_3 \cos(t\theta_1)$$

$$v'_4 = v_4 \cos(t\theta_2) - v_5 \sin(t\theta_2)$$

$$v'_5 = v_4 \sin(t\theta_2) + v_5 \cos(t\theta_2)$$

To express this entire system as a single matrix equation $\mathbf{v}' = R_t\mathbf{v}$ the coefficients for each input component dictate the matrix entries. Because v'_0 and v'_1 depend exclusively on v_0 and v_1 , the first two rows must contain zeros for all other columns. This strict independence between dimension pairs is what forces the general matrix into a block-diagonal structure.

Assembling the full 6×6 matrix multiplication demonstrates this decomposition:

$$\begin{bmatrix} v'_0 \\ v'_1 \\ v'_2 \\ v'_3 \\ v'_4 \\ v'_5 \end{bmatrix} = \begin{bmatrix} \cos(t\theta_0) & -\sin(t\theta_0) & 0 & 0 & 0 & 0 \\ \sin(t\theta_0) & \cos(t\theta_0) & 0 & 0 & 0 & 0 \\ 0 & 0 & \cos(t\theta_1) & -\sin(t\theta_1) & 0 & 0 \\ 0 & 0 & \sin(t\theta_1) & \cos(t\theta_1) & 0 & 0 \\ 0 & 0 & 0 & 0 & \cos(t\theta_2) & -\sin(t\theta_2) \\ 0 & 0 & 0 & 0 & \sin(t\theta_2) & \cos(t\theta_2) \end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{bmatrix}$$

Every off-diagonal block is zero, ensuring that each dimension pair is rotated independently within its own 2D plane without bleeding into the others. The frequency θ_i decreases for each subsequent block, so the first pair rotates rapidly while the final pair rotates very slowly.

The Odd-Dimension Edge Case ($d_{model} = 3$)

The toy model has $d_{model} = 3$, an odd number. RoPE operates on dimension pairs, so the three dimensions are partitioned as follows:

- **Dimensions 0 and 1** form a complete pair ($i = 0$). The rotation angle is $t \cdot \theta_0$.
- **Dimension 2** is unpaired. No second dimension exists to form the imaginary part of a complex number.

Standard practice in production implementations is to leave unpaired dimensions unrotated: the unpaired dimension passes through unchanged, as if multiplied by the 1×1 identity matrix. Every frontier model uses an even d_{model} (typically 128 per attention head), which avoids unpaired dimensions entirely. The toy model forces this edge case to the surface.

The full 3×3 rotation matrix R_t is block-diagonal:

$$R_t = \begin{bmatrix} \cos(t \cdot \theta_0) & -\sin(t \cdot \theta_0) & 0 \\ \sin(t \cdot \theta_0) & \cos(t \cdot \theta_0) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The upper-left 2×2 block rotates dimensions 0 and 1 by angle $t \cdot \theta_0$. The lower-right 1×1 block is the identity, leaving dimension 2 unchanged.

Expanding the full matrix-vector multiplication for a generic vector $\mathbf{v}$ demonstrates exactly how the unpaired dimension is preserved:

$$R_t \, \mathbf{v} = \begin{bmatrix} \cos(t \cdot \theta_0) & -\sin(t \cdot \theta_0) & 0 \\ \sin(t \cdot \theta_0) & \cos(t \cdot \theta_0) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} v_0 \cos(t\theta_0) - v_1 \sin(t\theta_0) \\ v_0 \sin(t\theta_0) + v_1 \cos(t\theta_0) \\ v_2 \end{bmatrix}$$

The first two components are mixed by the rotation, but the zeros in the third row isolate v_2 , allowing it to pass through untouched.

Computing the Angular Frequency

The rotation angle depends on the frequency θ_i , which is determined by the dimension pair index i and the total dimensions d_{model} . The formula for θ_i is:

$$\theta_i = \frac{1}{10000^{\,2i/d_{model}}}$$

For the toy model ($d_{model} = 3$), the only complete dimension pair is $i = 0$. Plugging these values into the frequency formula yields exactly 1:

$$\theta_0 = \frac{1}{10000^{\,0/3}} = \frac{1}{10000^0} = \frac{1}{1} = 1$$

The absolute position index t is simply a discrete integer ($0, 1, 2, 3$). RoPE translates this index into a geometric angle by using the frequency θ_i as a conversion factor. The frequency θ_i defines the rotation rate in **radians per position step** (where 1 radian is the standard mathematical unit for angles, equal to $\approx 57.3°$).

The total rotation angle is computed by multiplying the position t by this rate ($t \cdot \theta_i$). For the toy model's first dimension pair, the frequency is exactly $\theta_0 = 1$ radian per step. This means the total angle simplifies perfectly to $t \cdot 1 = t$ radians. The mathematical machinery literally reinterprets the token index integer as an angle.

This mapping dictates the precise rotations for the sequence:

- **Position 0**: $0 \text{ steps} \times 1 \text{ rad/step} = 0 \text{ radians}$ ($0°$).
- **Position 1**: $1 \text{ step} \times 1 \text{ rad/step} = 1 \text{ radian}$ ($\approx 57.3°$).
- **Position 2**: $2 \text{ steps} \times 1 \text{ rad/step} = 2 \text{ radians}$ ($\approx 114.6°$).
- **Position 3**: $3 \text{ steps} \times 1 \text{ rad/step} = 3 \text{ radians}$ ($\approx 171.9°$).

The angular frequency for the toy model's single dimension pair is exactly $\theta_0 = 1$ radian per position step. The rotation angle at each position simplifies to t radians: position 0 maps to 0 radians, position 1 to 1 radian, position 2 to 2 radians, and position 3 to 3 radians. The next chapter computes the explicit rotation matrix at each of these four positions and applies the rotations to every query and key vector from the toy model.

Chapter 6: Rotary Position Embeddings: Computing the Rotated Vectors

The preceding chapter derived the 3×3 rotation matrix for the toy model's odd-dimensional embedding space ($d_{model} = 3$). The upper-left 2×2 block rotates the complete dimension pair (dimensions 0 and 1) by the position-dependent angle $t \cdot \theta_0 = t$ radians, while the lower-right 1×1 identity block passes dimension 2 through unchanged. This chapter evaluates that matrix at every position in the toy sequence, proves its orthogonality, and applies the resulting rotations to every query and key vector.

Computing the Rotation Matrices

Each position $t \in \{0, 1, 2, 3\}$ has a specific 3×3 rotation matrix. Because the rotation angle is exactly t , the nine matrix entries are derived directly from $\cos(t)$ and $\sin(t)$.

Position $t = 0$ (rotation by 0 radians)

$$\cos(0) = 1.0000, \qquad \sin(0) = 0.0000$$

$$R_0 = \begin{bmatrix} 1.0000 & 0.0000 & 0 \\ 0.0000 & 1.0000 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

R_0 is the identity matrix. The first position in the sequence receives no rotation, which is consistent with the convention that position 0 is the reference orientation. All vectors at position 0 pass through unchanged.

Position $t = 1$ (rotation by 1 radian)

$$\cos(1) = 0.5403, \qquad \sin(1) = 0.8415$$

$$R_1 = \begin{bmatrix} 0.5403 & -0.8415 & 0 \\ 0.8415 & 0.5403 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The rotation by 1 radian is substantial: $\cos(1) \approx 0.54$ and $\sin(1) \approx 0.84$. Dimensions 0 and 1 are significantly mixed by this rotation, while dimension 2 is unchanged.

Position $t = 2$ (rotation by 2 radians)

$$\cos(2) = -0.4161, \qquad \sin(2) = 0.9093$$

$$R_2 = \begin{bmatrix} -0.4161 & -0.9093 & 0 \\ 0.9093 & -0.4161 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

At 2 radians, the cosine has crossed zero and turned negative. The $90°$ boundary ($\pi/2 \approx 1.57$ radians) dictates the sign of a dot product. If the angle between two vectors is less than $90°$, their dot product is positive; if it exceeds $90°$, the dot product is negative. Because $2 > 1.57$, the vector has rotated past this orthogonal boundary into the second quadrant. It now points away from its original orientation. The negative diagonal entries reflect this quadrant shift.

Position $t = 3$ (rotation by 3 radians)

$$\cos(3) = -0.9900, \qquad \sin(3) = 0.1411$$

$$R_3 = \begin{bmatrix} -0.9900 & -0.1411 & 0 \\ 0.1411 & -0.9900 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

At 3 radians ($\approx 171.9^\circ$), the rotation is nearly a full half-turn. The cosine is close to -1 , meaning dimension 0 is nearly inverted, and the sine is small and positive, meaning dimension 1 contributes only a small correction. The rotation matrix is close to $\mathrm{diag}(-1, -1, 1)$, which would be an exact 180° rotation at $t = \pi$.

Summary of Rotation Matrices

Position t	Angle (radians)	$\cos(t)$	$\sin(t)$
0	0.0000	1.0000	0.0000
1	1.0000	0.5403	0.8415
2	2.0000	−0.4161	0.9093
3	3.0000	−0.9900	0.1411

Every derived rotation matrix R_t is strictly orthogonal. This is an emergent structural property of the rotation matrix itself, rooted directly in the Pythagorean trigonometric identity $\cos^2(\alpha) + \sin^2(\alpha) = 1$.

Multiplying a 2×2 rotation block R by its transpose $R^\top$ explicitly demonstrates this property:

$$R^\top R = \begin{bmatrix} \cos(t\theta_i) & \sin(t\theta_i) \\ -\sin(t\theta_i) & \cos(t\theta_i) \end{bmatrix} \begin{bmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{bmatrix}$$

$$= \begin{bmatrix} \cos^2(t\theta_i) + \sin^2(t\theta_i) & -\cos(t\theta_i)\sin(t\theta_i) + \sin(t\theta_i)\cos(t\theta_i) \\ -\sin(t\theta_i)\cos(t\theta_i) + \cos(t\theta_i)\sin(t\theta_i) & (-\sin(t\theta_i))^2 + \cos^2(t\theta_i) \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

The off-diagonal terms cancel out completely, and the diagonal terms sum to 1, confirming that the matrix is mathematically orthogonal ($R_t^\top R_t = I$).

This orthogonality guarantees that the rotation preserves the lengths (norms) of the transformed vectors. Computing the squared norm of a rotated vector $R_t\mathbf{v}$ proves this preservation:

$$\|R_t\mathbf{v}\|^2 = (R_t\mathbf{v})^\top(R_t\mathbf{v})$$

$$= \mathbf{v}^\top(R_t^\top R_t)\mathbf{v}$$

Substituting the identity matrix I for $R_t^\top R_t$:

$$= \mathbf{v}^\top I\mathbf{v} = \mathbf{v}^\top\mathbf{v} = \|\mathbf{v}\|^2$$

Because the squared norms are equal, the magnitudes are strictly identical ($\|R_t\mathbf{v}\| = \|\mathbf{v}\|$).

This geometric preservation of magnitude provides a major structural advantage for RoPE over absolute additive encoding (like the sinusoidal encoding from Chapter 2). Additive encoding alters the original vector's length by directly adding a positional vector to it. RoPE only changes the vector's orientation, leaving its semantic magnitude completely intact.

Applying RoPE to the Query Vectors

The query vectors from Chapter 4 are rotated by their respective position matrices: $\mathbf{q}'_t = R_t\,\mathbf{q}_t$.

Position $t = 0$ (`The`): $\mathbf{q}_0 = [-0.08 \quad -0.13 \quad 0.21]^\top$

R_0 is the identity matrix, so the query vector passes through unchanged:

$$\mathbf{q}'_0 = R_0\,\mathbf{q}_0 = \begin{bmatrix} 1.0000 & 0.0000 & 0 \\ 0.0000 & 1.0000 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -0.08 \\ -0.13 \\ 0.21 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.0800} \\ \mathbf{-0.1300} \\ \mathbf{0.2100} \end{bmatrix}$$

Position $t = 1$ (`quick`): $\mathbf{q}_1 = [\ 0.35 \quad -0.17 \quad -0.27]^\top$

$$\mathbf{q}'_1 = R_1\,\mathbf{q}_1 = \begin{bmatrix} 0.5403 & -0.8415 & 0 \\ 0.8415 & 0.5403 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.35 \\ -0.17 \\ -0.27 \end{bmatrix} = \begin{bmatrix} \mathbf{0.3322} \\ \mathbf{0.2027} \\ \mathbf{-0.2700} \end{bmatrix}$$

Position $t = 2$ (`brown`): $\mathbf{q}_2 = [-0.09 \quad 0.47 \quad -0.07]^\top$

$$\mathbf{q}'_2 = R_2\,\mathbf{q}_2 = \begin{bmatrix} -0.4161 & -0.9093 & 0 \\ 0.9093 & -0.4161 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -0.09 \\ 0.47 \\ -0.07 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.3899} \\ \mathbf{-0.2774} \\ \mathbf{-0.0700} \end{bmatrix}$$

Position $t = 3$ (`fox`): $\mathbf{q}_3 = [-0.01 \quad -0.08 \quad 0.15]^\top$

$$\mathbf{q}_3' = R_3\,\mathbf{q}_3 = \begin{bmatrix} -0.9900 & -0.1411 & 0 \\ 0.1411 & -0.9900 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -0.01 \\ -0.08 \\ 0.15 \end{bmatrix} = \begin{bmatrix} \mathbf{0.0212} \\ \mathbf{0.0778} \\ \mathbf{0.1500} \end{bmatrix}$$

Rotated Query Vectors Summary

Position	Token	$\mathbf{q}_t$ (before RoPE)	$\mathbf{q}_t'$ (after RoPE)
0	`The`	$[-0.0800, -0.1300, \quad 0.2100\,]$	$[-0.0800, -0.1300, \quad 0.2100\,]$
1	`quick`	$[\ 0.3500, -0.1700, -0.2700\,]$	$[\ 0.3322, \quad 0.2027, -0.2700\,]$
2	`brown`	$[-0.0900, \quad 0.4700, -0.0700\,]$	$[-0.3899, -0.2774, -0.0700\,]$
3	`fox`	$[-0.0100, -0.0800, \quad 0.1500\,]$	$[\ 0.0212, \quad 0.0778, \quad 0.1500\,]$

The third component of every vector is unchanged (the identity block for the unpaired dimension). The first two components are rotated by increasing angles: 0, 1, 2, 3 radians. At position 0, no rotation occurs. At position 2, the rotation of 2 radians has reversed the sign of $q_2'[1]$ (from 0.47 to -0.2774) and driven $q_2'[0]$ more negative (from -0.09 to -0.3899). These are not additive modifications; the vector has been geometrically reoriented in the 2D plane formed by dimensions 0 and 1.

Applying RoPE to the Key Vectors

The key vectors from Chapter 4 are rotated identically: $\mathbf{k}_t' = R_t\,\mathbf{k}_t$.

Position $t = 0$ (`The`): $\mathbf{k}_0 = [-0.15 \quad 0.10 \quad -0.04]^\top$

$$\mathbf{k}_0' = R_0\,\mathbf{k}_0 = \begin{bmatrix} 1.0000 & 0.0000 & 0 \\ 0.0000 & 1.0000 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -0.15 \\ 0.10 \\ -0.04 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.1500} \\ \mathbf{0.1000} \\ \mathbf{-0.0400} \end{bmatrix}$$

Position $t = 1$ (`quick`): $\mathbf{k}_1 = [\ 0.24 \quad 0.11 \quad -0.32]^\top$

$$\mathbf{k}_1' = R_1\,\mathbf{k}_1 = \begin{bmatrix} 0.5403 & -0.8415 & 0 \\ 0.8415 & 0.5403 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.24 \\ 0.11 \\ -0.32 \end{bmatrix} = \begin{bmatrix} \mathbf{0.0371} \\ \mathbf{0.2614} \\ \mathbf{-0.3200} \end{bmatrix}$$

Position $t = 2$ (`brown`): $\mathbf{k}_2 = [\ 0.16 \quad -0.17 \quad 0.36]^\top$

$$\mathbf{k}_2' = R_2\,\mathbf{k}_2 = \begin{bmatrix} -0.4161 & -0.9093 & 0 \\ 0.9093 & -0.4161 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.16 \\ -0.17 \\ 0.36 \end{bmatrix} = \begin{bmatrix} \mathbf{0.0880} \\ \mathbf{0.2162} \\ \mathbf{0.3600} \end{bmatrix}$$

Position $t = 3$ (`fox`): $\mathbf{k}_3 = [-0.06 \quad 0.11 \quad -0.05]^\top$

$$\mathbf{k}'_3 = R_3\,\mathbf{k}_3 = \begin{bmatrix} -0.9900 & -0.1411 & 0 \\ 0.1411 & -0.9900 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -0.06 \\ 0.11 \\ -0.05 \end{bmatrix} = \begin{bmatrix} \mathbf{0.0439} \\ \mathbf{-0.1174} \\ \mathbf{-0.0500} \end{bmatrix}$$

Rotated Key Vectors Summary

Position	Token	$\mathbf{k}_t$ (before RoPE)	$\mathbf{k}'_t$ (after RoPE)
0	`The`	$[-0.1500, \quad 0.1000, -0.0400\,]$	$[-0.1500, \quad 0.1000, -0.0400\,]$
1	`quick`	$[\ 0.2400, \quad 0.1100, -0.3200\,]$	$[\ 0.0371, \quad 0.2614, -0.3200\,]$
2	`brown`	$[\ 0.1600, -0.1700, \quad 0.3600\,]$	$[\ 0.0880, \quad 0.2162, \quad 0.3600\,]$
3	`fox`	$[-0.0600, \quad 0.1100, -0.0500\,]$	$[\ 0.0439, -0.1174, -0.0500\,]$

What Has Been Accomplished

The RoPE operation has injected positional information into every query and key vector through geometric rotation, without adding any vector to the embeddings. The key structural properties of this encoding:

Nothing is added. Unlike sinusoidal PE (Chapter 3), RoPE does not add a positional vector to the embedding. The embedding tensor X is untouched. The positional signal exists only in the rotated orientation of the query and key vectors, not in their magnitudes.

Norms are preserved. Because R_t is an orthogonal matrix, $\|\mathbf{q}'_t\| = \|\mathbf{q}_t\|$ and $\|\mathbf{k}'_t\| = \|\mathbf{k}_t\|$. The semantic magnitude of each vector is unaltered; only its direction in the 2D subspace formed by dimensions 0 and 1 has changed.

Each position receives a distinct rotation. Position 0 is the reference orientation (no rotation). Position 1 is rotated by 1 radian. Position 2 by 2 radians. Position 3 by 3 radians. The rotation angle increases linearly with position, producing a unique geometric orientation for each position index.

The unpaired dimension carries no positional signal. Dimension 2 passes through the identity for all positions. In the toy model with $d_{model} = 3$, two out of three dimensions carry positional information. In production models with even d_{model}, all dimensions are paired and all carry positional signal.

The rotated query and key vectors are now ready to be consumed directly by the Transformer's attention mechanism. It is critical to understand that the Transformer never "decodes" these rotated vectors back into their original absolute positions. Instead, the positional information is structurally baked into the vectors, and the attention mechanism computes the dot product ($(\mathbf{q}'_m)^\top \mathbf{k}'_n$) between them. Because the Value vectors (V) are not rotated by RoPE, the positional information influences *only* the attention scores (which tokens pay attention to which), not the semantic content that is ultimately extracted.

The next chapter proves the central geometric property of RoPE: how the dot product of two rotated vectors $(\mathbf{q}'_m)^\top \mathbf{k}'_n$ mathematically collapses to depend exclusively on their relative distance ($m - n$), satisfying the relative positional requirement established in Chapter 4.

Chapter 7: The Relative-Distance Property of RoPE

The preceding chapters derived the RoPE rotation matrix from its complex-number foundations and applied it at every position to all query and key vectors from the toy model. The resulting rotated vectors $\mathbf{q}'_t = R_t \mathbf{q}_t$ and $\mathbf{k}'_t = R_t \mathbf{k}_t$ encode position through geometric orientation rather than additive modification. But the derivation stopped short of proving the property that motivated the entire construction: that the dot product of rotated vectors depends only on content and relative position.

This chapter delivers that proof. The rotation composition identity $R_m^\top R_n = R_{n-m}$ is derived from the angle-addition identities for sine and cosine, verified numerically using the toy model vectors, and applied to compute the full post-RoPE relevance score matrix. The result is then compared with the pre-RoPE matrix from Chapter 4 to demonstrate that RoPE has injected position-dependent structure into the relevance scores.

The Central Property

For any two positions m and n, the dot product of the rotated query at position m and the rotated key at position n satisfies:

$$\mathbf{q}'^\top_m \mathbf{k}'_n = (R_m \mathbf{q}_m)^\top (R_n \mathbf{k}_n) = \mathbf{q}_m^\top R_m^\top R_n \, \mathbf{k}_n = \mathbf{q}_m^\top R_{n-m} \, \mathbf{k}_n$$

The first equality substitutes the definition of rotated vectors. The second equality uses the transpose property of matrix products: $(R_m \mathbf{q}_m)^\top = \mathbf{q}_m^\top R_m^\top$. The third equality is the claim: the matrix product $R_m^\top R_n$ collapses to a single rotation matrix R_{n-m} whose angle depends only on the offset $n - m$.

If this claim holds, then the relevance score between positions m and n is:

$$\mathbf{q}'^\top_m \mathbf{k}'_n = \mathbf{q}_m^\top R_{n-m} \, \mathbf{k}_n$$

The right-hand side depends on the content vectors $\mathbf{q}_m$ and $\mathbf{k}_n$ and on the relative offset $\Delta = n - m$, but not on the absolute positions m and n individually. This is exactly the relative position requirement stated in Chapter 4.

The claim rests on a single algebraic fact: the rotation composition identity $R_m^\top R_n = R_{n-m}$. The remainder of this section proves it from first principles.

Proving the Rotation Composition Identity

The Transpose of a Rotation Matrix

A rotation matrix R_t for angle $\alpha = t \cdot \theta$ (dropping the subscript i on θ for notational clarity, since the proof applies identically to each dimension pair) has the form:

$$R_\alpha = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix}$$

The transpose is obtained by reflecting across the main diagonal:

$$R_\alpha^\top = \begin{bmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{bmatrix}$$

Because cosine is an even function ($\cos(-\alpha) = \cos\alpha$) and sine is an odd function ($\sin(-\alpha) = -\sin\alpha$), this transpose is identical to a rotation by $-\alpha$:

$$R_{-\alpha} = \begin{bmatrix} \cos(-\alpha) & -\sin(-\alpha) \\ \sin(-\alpha) & \cos(-\alpha) \end{bmatrix} = \begin{bmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{bmatrix} = R_\alpha^\top$$

The transpose of a rotation matrix is its inverse: rotating by angle α and then by $-\alpha$ returns to the original orientation. Equivalently, $R_\alpha^\top R_\alpha = I$.

Multiplying Two Rotation Matrices

The composition identity states that rotating by $-\alpha$ and then by β is equivalent to a single rotation by $\beta - \alpha$. Written as a matrix equation:

$$R_\alpha^\top R_\beta = R_{-\alpha} \cdot R_\beta = R_{\beta-\alpha}$$

To prove this, the product of $R_{-\alpha}$ and R_β is computed directly:

$$R_{-\alpha} \cdot R_\beta = \begin{bmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{bmatrix} \begin{bmatrix} \cos\beta & -\sin\beta \\ \sin\beta & \cos\beta \end{bmatrix} = \begin{bmatrix} \cos\alpha\cos\beta + \sin\alpha\sin\beta & -\cos\alpha\sin\beta + \sin\alpha\cos\beta \\ -\sin\alpha\cos\beta + \cos\alpha\sin\beta & \sin\alpha\sin\beta + \cos\alpha\cos\beta \end{bmatrix}$$

Applying the Angle-Addition Identities

The angle-addition identities for cosine and sine are:

$$\cos(\beta - \alpha) = \cos\beta\cos\alpha + \sin\beta\sin\alpha$$

$$\sin(\beta - \alpha) = \sin\beta\cos\alpha - \cos\beta\sin\alpha$$

Each entry of the product matrix is now matched to one of these identities:

Entry $(0, 0)$:

$$\cos\alpha\cos\beta + \sin\alpha\sin\beta = \cos(\beta - \alpha)$$

Entry $(0, 1)$:

$$-\cos\alpha\sin\beta + \sin\alpha\cos\beta = -(\cos\alpha\sin\beta - \sin\alpha\cos\beta) = -\sin(\beta - \alpha)$$

Entry $(1, 0)$:

$$-\sin\alpha\cos\beta+\cos\alpha\sin\beta=\cos\alpha\sin\beta-\sin\alpha\cos\beta=\sin(\beta-\alpha)$$

Entry $(1,1)$:

$$\sin\alpha\sin\beta+\cos\alpha\cos\beta=\cos(\beta-\alpha)$$

The Result

Substituting the angle-addition identities into the product matrix:

$$R_{-\alpha}\cdot R_{\beta}=\begin{bmatrix}\cos(\beta-\alpha) & -\sin(\beta-\alpha)\\ \sin(\beta-\alpha) & \cos(\beta-\alpha)\end{bmatrix}=R_{\beta-\alpha}$$

This is the standard 2×2 rotation matrix for angle $\beta-\alpha$. Therefore:

$$R_{\alpha}^{\top}R_{\beta}=R_{\beta-\alpha}$$

Setting $\alpha=m\theta_i$ and $\beta=n\theta_i$ for any dimension pair i yields the rotation matrix for the relative angle $(n-m)\theta_i$:

$$R_{m\theta_i}^{\top}R_{n\theta_i}=R_{(n-m)\theta_i}$$

Because this relative-angle identity holds independently for every block on the diagonal, the full position-dependent matrix satisfies:

$$R_m^{\top}R_n=R_{n-m}$$

The product of the transposed rotation at position m and the rotation at position n is a single rotation whose angle depends only on the offset $n-m$. This is the rotation composition identity, and it is exact (not an approximation), because it follows from the algebraic properties of sine and cosine.

For the full 3×3 block-diagonal matrix used in the toy model, the identity applies to the upper-left 2×2 block, and the lower-right 1×1 identity block trivially satisfies $1\cdot 1=1$. The composition identity therefore holds for the full 3×3 rotation matrix at every position.

Numerical Verification with the Toy Model

The algebraic proof establishes the identity in general. This section verifies it concretely using the rotated vectors from Chapter 6. The pair chosen for verification is `quick` (position 1) and `fox` (position 3), with offset $\Delta=n-m=2$.

Method 1: Direct Dot Product of Rotated Vectors ($\mathbf{q}_1'^{\top}\mathbf{k}_3'$)

The rotated vectors at positions 1 and 3, computed in Chapter 6:

$$\mathbf{q}_1' = \begin{bmatrix} 0.3322 \\ 0.2027 \\ -0.2700 \end{bmatrix}, \qquad \mathbf{k}_3' = \begin{bmatrix} 0.0439 \\ -0.1174 \\ -0.0500 \end{bmatrix}$$

The dot product is computed term by term:

$$\mathbf{q}_1'^{\top}\mathbf{k}_3' = (0.3322)(0.0439) + (0.2027)(-0.1174) + (-0.2700)(-0.0500) = \mathbf{0.0043}$$

Method 2: Relative Rotation Applied to Unrotated Vectors ($\mathbf{q}_1^{\top} R_2\, \mathbf{k}_3$)

The relative position requirement predicts that the same score can be obtained by applying the relative rotation $R_{\Delta} = R_{3-1} = R_2$ to the unrotated key vector, then taking the dot product with the unrotated query vector.

The unrotated vectors from Chapter 4:

$$\mathbf{q}_1 = \begin{bmatrix} 0.35 \\ -0.17 \\ -0.27 \end{bmatrix}, \qquad \mathbf{k}_3 = \begin{bmatrix} -0.06 \\ 0.11 \\ -0.05 \end{bmatrix}$$

First, compute $R_2\mathbf{k}_3$. The rotation matrix at $t = 2$ (from Chapter 6):

$$R_2 = \begin{bmatrix} -0.4161 & -0.9093 & 0 \\ 0.9093 & -0.4161 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Applying R_2 to $\mathbf{k}_3$, component by component:

$$(R_2\mathbf{k}_3)[0] = (-0.4161)(-0.06) + (-0.9093)(0.11) = \mathbf{-0.0751}$$

$$(R_2\mathbf{k}_3)[1] = (0.9093)(-0.06) + (-0.4161)(0.11) = \mathbf{-0.1003}$$

$$(R_2\mathbf{k}_3)[2] = -0.05 \qquad \textbf{(identity, unpaired dimension)}$$

$$R_2\mathbf{k}_3 = \begin{bmatrix} -0.0751 \\ -0.1003 \\ -0.0500 \end{bmatrix}$$

Now compute $\mathbf{q}_1^{\top}(R_2\mathbf{k}_3)$:

$$\mathbf{q}_1^{\top}(R_2\mathbf{k}_3) = (0.35)(-0.0751) + (-0.17)(-0.1003) + (-0.27)(-0.0500) = \mathbf{0.0043}$$

Verification

Both methods produce the same scalar:

$$\mathbf{q}_1'^{\top}\mathbf{k}_3' = \mathbf{0.0043} = \mathbf{q}_1^{\top} R_2\, \mathbf{k}_3$$

The direct dot product of the rotated vectors (Method 1) equals the dot product obtained by applying only the relative rotation R_2 to the unrotated key (Method 2). The absolute positions 1 and 3 have disappeared from the computation; only the offset $\Delta = 2$ remains.

This is the relative-distance property in action. If `quick` appeared at position 5 and `fox` at position 7 (same content, same offset $\Delta = 2$), the same rotation R_2 would produce the same relevance score. The score 0.0043 is determined by the content of `quick` and `fox` and by their separation of 2 positions, regardless of where in the sequence that separation occurs.

The Full Post-RoPE Relevance Score Matrix

The dot product $A'_{ij} = \mathbf{q}'^{\top}_i \mathbf{k}'_j$ is now computed for all 16 pairs of positions using the rotated vectors from Chapter 6:

$$\mathbf{q}'_0 = \begin{bmatrix} -0.0800 \\ -0.1300 \\ 0.2100 \end{bmatrix}, \quad \mathbf{q}'_1 = \begin{bmatrix} 0.3322 \\ 0.2027 \\ -0.2700 \end{bmatrix}, \quad \mathbf{q}'_2 = \begin{bmatrix} -0.3899 \\ -0.2774 \\ -0.0700 \end{bmatrix}, \quad \mathbf{q}'_3 = \begin{bmatrix} 0.0212 \\ 0.0778 \\ 0.1500 \end{bmatrix}$$

$$\mathbf{k}'_0 = \begin{bmatrix} -0.1500 \\ 0.1000 \\ -0.0400 \end{bmatrix}, \quad \mathbf{k}'_1 = \begin{bmatrix} 0.0371 \\ 0.2614 \\ -0.3200 \end{bmatrix}, \quad \mathbf{k}'_2 = \begin{bmatrix} 0.0880 \\ 0.2162 \\ 0.3600 \end{bmatrix}, \quad \mathbf{k}'_3 = \begin{bmatrix} 0.0439 \\ -0.1174 \\ -0.0500 \end{bmatrix}$$

Row 0 (`The` queries each position):

$$A'_{00} = (-0.0800)(-0.1500) + (-0.1300)(0.1000) + (0.2100)(-0.0400) = \mathbf{-0.0094}$$

$$A'_{01} = (-0.0800)(0.0371) + (-0.1300)(0.2614) + (0.2100)(-0.3200) = \mathbf{-0.1041}$$

$$A'_{02} = (-0.0800)(0.0880) + (-0.1300)(0.2162) + (0.2100)(0.3600) = \mathbf{0.0405}$$

$$A'_{03} = (-0.0800)(0.0439) + (-0.1300)(-0.1174) + (0.2100)(-0.0500) = \mathbf{0.0012}$$

Row 1 (`quick` queries each position):

$$A'_{10} = (0.3322)(-0.1500) + (0.2027)(0.1000) + (-0.2700)(-0.0400) = \mathbf{-0.0188}$$

$$A'_{11} = (0.3322)(0.0371) + (0.2027)(0.2614) + (-0.2700)(-0.3200) = \mathbf{0.1517}$$

$$A'_{12} = (0.3322)(0.0880) + (0.2027)(0.2162) + (-0.2700)(0.3600) = \mathbf{-0.0241}$$

$$A'_{13} = (0.3322)(0.0439) + (0.2027)(-0.1174) + (-0.2700)(-0.0500) = \mathbf{0.0043}$$

Row 2 (`brown` queries each position):

$$A'_{20} = (-0.3899)(-0.1500) + (-0.2774)(0.1000) + (-0.0700)(-0.0400) = \mathbf{0.0335}$$

$$A'_{21} = (-0.3899)(0.0371) + (-0.2774)(0.2614) + (-0.0700)(-0.3200) = \mathbf{-0.0646}$$

$$A'_{22} = (-0.3899)(0.0880) + (-0.2774)(0.2162) + (-0.0700)(0.3600) = \mathbf{-0.1195}$$

$$A'_{23} = (-0.3899)(0.0439) + (-0.2774)(-0.1174) + (-0.0700)(-0.0500) = \mathbf{0.0190}$$

Row 3 (`fox` queries each position):

$$A'_{30} = (0.0212)(-0.1500) + (0.0778)(0.1000) + (0.1500)(-0.0400) = \mathbf{-0.0014}$$

$$A'_{31} = (0.0212)(0.0371) + (0.0778)(0.2614) + (0.1500)(-0.3200) = \mathbf{-0.0269}$$

$$A'_{32} = (0.0212)(0.0880) + (0.0778)(0.2162) + (0.1500)(0.3600) = \mathbf{0.0727}$$

$$A'_{33} = (0.0212)(0.0439) + (0.0778)(-0.1174) + (0.1500)(-0.0500) = \mathbf{-0.0157}$$

The Complete Post-RoPE Relevance Score Matrix

$$A' = \begin{bmatrix} -0.0094 & -0.1041 & 0.0405 & 0.0012 \\ -0.0188 & 0.1517 & -0.0241 & 0.0043 \\ 0.0335 & -0.0646 & -0.1195 & 0.0190 \\ -0.0014 & -0.0269 & 0.0727 & -0.0157 \end{bmatrix}$$

Comparison with the Pre-RoPE Matrix

The pre-RoPE relevance score matrix from Chapter 4:

$$A = \begin{bmatrix} -0.0094 & -0.1007 & 0.0849 & -0.0200 \\ -0.0587 & 0.1517 & -0.0123 & -0.0262 \\ 0.0633 & 0.0525 & -0.1195 & 0.0606 \\ -0.0125 & -0.0592 & 0.0660 & -0.0157 \end{bmatrix}$$

Several structural observations emerge from comparing A and A'.

The diagonal is unchanged. The diagonal entries $A'_{ii} = A_{ii}$ for all i:

Position	A_{ii}	A'_{ii}
0 (`The`)	-0.0094	-0.0094
1 (`quick`)	0.1517	0.1517
2 (`brown`)	-0.1195	-0.1195
3 (`fox`)	-0.0157	-0.0157

This is a direct consequence of the rotation composition identity. For the diagonal, $m = n$, so the relative rotation is $R_{n-n} = R_0 = I$ (the identity matrix). The self-relevance score $\mathbf{q}_i^\top R_0 \mathbf{k}_i = \mathbf{q}_i^\top \mathbf{k}_i$ is the same as the unrotated dot product. A token's relevance to itself is unaffected by its absolute position, because the relative offset to itself is always zero.

Off-diagonal entries have changed. Every off-diagonal entry differs between A and A':

Token Pair	Offset Δ	A_{ij} (pre-RoPE)	A'_{ij} (post-RoPE)
`The` → `quick`	1	-0.1007	-0.1041
`The` → `brown`	2	0.0849	0.0405
`The` → `fox`	3	-0.0200	0.0012
`quick` → `The`	-1	-0.0587	-0.0188
`quick` → `brown`	1	-0.0123	-0.0241
`quick` → `fox`	2	-0.0262	0.0043
`brown` → `The`	-2	0.0633	0.0335
`brown` → `quick`	-1	0.0525	-0.0646
`brown` → `fox`	1	0.0606	0.0190
`fox` → `The`	-3	-0.0125	-0.0014
`fox` → `quick`	-2	-0.0592	-0.0269
`fox` → `brown`	-1	0.0660	0.0727

The rotation has modified the relevance landscape. Consider the pair `The` → `brown` (positions 0 and 2): the pre-RoPE score 0.0849 drops to 0.0405 after RoPE. The rotation R_2 applied to the relative offset $\Delta = 2$ has reduced the apparent relevance. Meanwhile, `quick` → `fox` (positions 1 and 3, same offset $\Delta = 2$) shifts from -0.0262 to 0.0043, crossing from negative to positive. The same relative rotation R_2 produces different effects on different content pairs, because the rotation interacts with the content vectors differently. This is by design: the score depends on both content and relative position.

Permutation invariance is broken. The pre-RoPE matrix A satisfies $A_{rev} = PAP^\top$ (Chapter 4): reversing the input sequence rearranges the scores but does not change them. The post-RoPE matrix A' does not satisfy this property. In the reversed sequence `fox brown quick The`, the token `fox` would occupy position 0 and `The` would occupy position 3. Their relative offset would be $\Delta = 3 - 0 = 3$ (not $\Delta = -3$ as in the original sequence). Because $R_3 \neq R_{-3}$, the relevance score between `fox` and `The` would differ from the original ordering. RoPE has broken permutation invariance through the mechanism of position-dependent rotation.

Grounding the Result

The relevance score between `quick` (position 1) and `fox` (position 3) is $A'_{13} = 0.0043$. This score depends on the semantic content of `quick` and `fox` (encoded in the query vector $\mathbf{q}_1$ and key vector $\mathbf{k}_3$) and on the offset $\Delta = 2$ between their positions (encoded in the rotation R_2). It does not depend on the absolute position indices 1 and 3.

If `quick` appeared at position 5 and `fox` at position 7, the content vectors $\mathbf{q}$ and $\mathbf{k}$ would be identical (same tokens, same projection matrices), and the relative rotation would be $R_{7-5} = R_2$ (same offset). The relevance score would be $\mathbf{q}^\top R_2 \mathbf{k} = 0.0043$, unchanged. If `quick` appeared at position 100 and `fox` at position 102, the score would again be 0.0043. The absolute positions are irrelevant; only the offset $\Delta = 2$ between the two tokens determines the rotational transformation.

This is the relative position requirement from Chapter 4, now satisfied by algebraic construction.

Why This Property Matters

The relative-distance property of RoPE has three consequences that distinguish it from additive positional encoding.

Relative relationships, not absolute labels. The model learns how tokens at different separations interact, not what happens at position 47 specifically. A verb three positions after its subject receives the same relative rotation regardless of whether the subject appears at position 0, position 50, or position 500. This aligns with the linguistic observation that syntactic and semantic relationships between tokens typically depend on their relative displacement, not on their absolute indices within the sequence.

Length generalization. Because the rotation R_Δ depends only on the offset Δ, the mechanism encounters a rotation for offset $\Delta = 200$ during inference even if the longest training sequence had length 150 (provided $\Delta = 200$ arose in some training context). Sinusoidal absolute PE, by contrast, assigns absolute position encodings that degrade for unseen positions. RoPE's relative encoding extends more naturally to sequence lengths beyond the training distribution, though practical generalization still requires careful frequency scaling (the subject of Chapter 8).

Exact algebraic identity. The property $R_m^\top R_n = R_{n-m}$ is not an approximation learned during training, nor an empirical observation that holds approximately for most inputs. It is an algebraic identity that follows from the angle-addition formulas for sine and cosine. Every rotated dot product in every attention head at every layer satisfies this property exactly, because it is a consequence of the mathematical structure of rotation matrices, not of the learned parameters. The model's positional encoding is correct by construction.

The rotation composition identity, proved from the angle-addition identities and verified numerically with the toy model, completes the mathematical foundation of RoPE. The next chapter surveys the broader landscape of positional encoding mechanisms, including learned absolute encodings, relative position biases, ALiBi, iRoPE, and frequency-scaling extensions for long-context generalization.

Chapter 8: The Modern Landscape and Frontier Extensions

The preceding six chapters constructed two positional encoding mechanisms from first principles. Chapters 2 and 3 derived the sinusoidal absolute encoding of Vaswani et al. (2017), added it to the embedding tensor, and demonstrated that it breaks permutation invariance. Chapters 4, 5, and 6 introduced the query-key framework, derived Rotary Position Embeddings (Su et al., 2021), and proved that the resulting dot product depends only on content and relative position offset. Both mechanisms received full numerical treatment through the toy model.

These two are not the full picture. Between the original Transformer and the frontier models deployed today, several alternative approaches to positional encoding were proposed, each motivated by a specific shortcoming of the mechanisms that preceded it. This chapter traces that evolution conceptually, explaining what each mechanism does, why it was designed that way, and where it falls short.

Learned Absolute Positional Embeddings

The sinusoidal formula from Chapter 2 computes positional vectors from a fixed mathematical function. A simpler alternative is to skip the formula entirely and let the model learn its own positional vectors during training.

In this approach, a separate embedding matrix is allocated with one row per position (up to some maximum sequence length). Each row is a trainable vector, initialized randomly and updated through backpropagation alongside the token embeddings and all other model parameters. At runtime, the vector for position t is looked up from this matrix and added element-wise to the token embedding at that position, exactly as the sinusoidal vectors are added in Chapter 3.

The appeal is maximum flexibility. If the optimal positional encoding for a given task happens to resemble sinusoidal patterns, the model is free to learn that structure. If the optimal encoding has an entirely different geometry, the model is not constrained to trigonometric shapes. The encoding adapts to whatever the training data rewards.

This mechanism was used in BERT (Devlin et al., 2018) with a maximum length of 512 tokens and in GPT-2 (Radford et al., 2019) with a maximum length of 1,024 tokens.

The fundamental limitation is a hard ceiling on sequence length. The embedding matrix has a fixed number of rows, determined before training begins. Position 513 in a BERT model has no entry in the matrix and cannot be represented. Extending to longer sequences requires either retraining the model with a larger matrix or interpolating between existing entries (an imprecise workaround). As context lengths grew from 512 to 128,000 and beyond, this ceiling became untenable.

A subtler limitation is the absence of structural guarantees. None of the five design constraints from Chapter 2 (uniqueness, boundedness, determinism, smoothness, relative representability) are satisfied by construction. Whether adjacent positions receive similar vectors, whether the encoding captures relative offsets, whether the values stay bounded, all depend on what the optimizer happens to discover during training. The sinusoidal formula guarantees these properties mathematically. Learned embeddings hope that training will produce them empirically.

Relative Positional Encoding

The sinusoidal formula and learned embeddings both share a structural assumption: position information is injected into the embedding tensor *before* the attention computation begins. Each position receives a fixed vector (computed or learned), that vector is added to the token embedding, and the combined representation flows into the query and key projections. As Chapter 3 demonstrated, this entangles the content signal and the positional signal in the same vector space, forcing the model to disentangle them.

Shaw et al. (2018) proposed a different approach: inject position information directly into the attention scores, at the point where the model is actually computing relationships between tokens.

The mechanism works by adding a learned bias to the attention score between every pair of positions. Crucially, this bias is indexed by the *relative offset* between the two positions, not by their absolute indices. Two token pairs separated by the same number of positions receive the same bias, regardless of where they sit in the sequence. The bias is a learned vector (one for each possible offset within a clipping window), so the model can learn arbitrary relationships between tokens at different separations.

The key insight is that attention scores should reflect how far apart two tokens are, not where each token sits in absolute terms. A verb three positions after its subject should receive a similar positional signal whether the subject appears at position 0, position 50, or position 500. By parameterizing the bias with the offset rather than the individual positions, this mechanism structurally encodes that insight.

This approach influenced the positional encoding designs in Transformer-XL (Dai et al., 2019) and the T5 model (Raffel et al., 2020).

The limitations mirror those of learned absolute embeddings in a different form. The bias vectors are learned parameters, so the encoding varies across training runs and provides no structural guarantees about smoothness or boundedness. The clipping window imposes a maximum relative distance: offsets beyond the window share a single bias vector, meaning the model cannot distinguish between tokens 100 positions apart and tokens 200 positions apart. And the relative relationship is stored in a lookup table of learned vectors rather than emerging from algebraic structure (as it does in RoPE, where the relative-distance property follows from the angle-addition identities).

ALiBi (Attention with Linear Biases)

Press et al. (2022) took the idea of modifying attention scores and pushed it to its minimalist extreme: no positional encoding on the embeddings, no positional encoding on the query or key projections, and no learned parameters of any kind. Instead, a fixed linear penalty is subtracted from each attention score, proportional to the distance between the two tokens.

Tokens close together receive a small penalty (high attention is easy). Tokens far apart receive a large penalty (high attention is difficult). Different attention heads use different penalty slopes, set as fixed constants determined entirely by the number of heads. Some heads apply a steep penalty, creating a sharp locality bias that focuses attention on nearby tokens. Other heads apply a gentle penalty, allowing attention to spread across the full sequence.

The design motivation is simplicity and length generalization. Because the penalty is a linear function of distance, it extrapolates naturally to distances unseen during training: the penalty for a distance of 10,000 is simply 10,000 times the slope, with no periodic structure to break down. The mechanism introduces zero additional parameters (the slopes are predetermined constants) and leaves the content pathway completely untouched.

ALiBi was adopted in BLOOM (BigScience, 2022) and MPT (MosaicML, 2023).

The shortcoming is rigidity. The assumption that relevance decays linearly with distance is a strong structural prior that does not match the complexity of natural language. A verb may be highly relevant to its subject 20 tokens away but irrelevant to the adjacent comma. A pronoun at position 50 may refer to a noun at position 5, making that distant token more relevant than every intervening token. ALiBi cannot express these patterns; it can only penalize distance uniformly. Subsequent frontier models (GPT-4, Claude, Gemini, Llama 3) adopted RoPE instead, which encodes relative position through geometric rotation rather than scalar penalty, allowing the content of the tokens to interact with the positional signal in richer ways.

iRoPE (Interleaved RoPE)

Standard RoPE models apply position-dependent rotations to query and key vectors in every attention layer. iRoPE (Meta, 2025), the architecture used in Llama 4, makes a selective choice: only some layers apply RoPE, while the remaining layers apply no positional encoding at all.

The motivation comes from observing that not all attention patterns require positional information. Some attention heads learn syntactic patterns (subject-verb agreement across specific relative positions, clause boundaries at characteristic distances) where positional information is essential. Other heads learn semantic patterns (thematic similarity, coreference, topical grouping) where the content of the tokens matters and their positions are irrelevant. Applying RoPE to every layer forces all attention heads to process position-rotated vectors, even in heads where the rotation adds noise to an otherwise clean semantic signal.

By alternating RoPE layers with position-free layers, the architecture provides both pathways. The RoPE layers encode explicit relative position. The position-free layers compute attention based purely on content, relying on the model's ability to infer any necessary positional context from the surrounding RoPE layers (through residual connections that propagate information between layers). The result is a model that can dedicate some of its attention capacity to position-dependent relationships and the rest to position-agnostic relationships, rather than forcing every layer to do both simultaneously.

A secondary benefit is computational efficiency during inference. Position-free layers do not require computing or applying rotation matrices, and their key-value caches do not depend on position, enabling more flexible caching strategies during autoregressive generation.

RoPE Extensions for Long Context

The relative-distance property proved in Chapter 7 holds at any sequence length: the rotation composition identity $R_m^\top R_n = R_{n-m}$ is an algebraic fact, independent of how large m and n become. But the model's *learned attention patterns* are not algebraic identities. They are statistical regularities optimized over the rotation angles encountered during training. When a model trained on sequences of length 4,096 encounters a sequence of length 32,000, the rotation angles at positions beyond 4,096 fall outside the training distribution. The mathematical structure is intact, but the model has never learned to interpret those angles.

This is the length extrapolation problem, and several approaches have been proposed to address it.

Position Interpolation (Chen et al., 2023) takes the most straightforward approach: scale all position indices by a compression factor so that the extended sequence maps onto the same range of rotation angles the model saw during training. A model trained on length 4,096 that needs to handle length 16,384 divides every position index by 4, so position 16,384 produces the same rotation angle that position 4,096 produced during training. The cost is reduced positional resolution: positions that were previously one unit apart now appear one-quarter of a unit apart, making it harder for the model to distinguish fine-grained positional differences.

NTK-aware scaling (Bloc97, 2023) observes that uniform compression damages some dimensions more than others. High-frequency dimensions (which capture fine-grained positional distinctions, like the difference between position 5 and position 6) are more sensitive to compression than low-frequency dimensions (which capture coarse positional structure, like the difference between position 0 and position 500). NTK-aware scaling applies different compression factors to different frequency dimensions: high-frequency dimensions are left mostly unchanged, preserving fine-grained resolution, while low-frequency dimensions absorb most of the compression, extending the effective range of the encoding. The interpolation pressure is concentrated on the dimensions that can absorb it.

YaRN (Peng et al., 2023) extends this idea further by partitioning all frequency dimensions into three groups (high-frequency, medium-frequency, low-frequency) and applying a different scaling strategy to each. High-frequency dimensions receive no modification. Low-frequency dimensions receive full interpolation. Medium-frequency dimensions receive a smooth blend between the two extremes. This fine-grained partitioning, combined with an attention-score temperature adjustment to compensate for the changed variance, produces the most precise frequency management of the three approaches. YaRN was adopted by DeepSeek V3 and R1 for extending RoPE to context lengths of 128,000 tokens and beyond.

All three extensions preserve the algebraic structure of RoPE. The rotation composition identity holds regardless of the specific frequency values, because the identity follows from the angle-addition properties of sine and cosine, not from the particular angles used. Modifying the frequencies changes the rotation angles but does not invalidate the relative-distance property. The extensions are adjustments to the frequency tuning, not alterations to the mathematical mechanism.

Final Thoughts

Positional encoding solves a problem that the architecture itself created. The Transformer processes all tokens simultaneously rather than sequentially, which gives it parallelism and scalability, but strips away the one thing that recurrent networks got for free: the knowledge of which token came first. Every mechanism in this series, from sinusoidal addition to geometric rotation to linear penalty, is an attempt to restore that lost ordering information without sacrificing the parallelism that made the Transformer worth building in the first place.

The common thread across every mechanism is a single goal: inject enough positional structure that the model can distinguish token order, without distorting the semantic content that makes each token meaningful. The sinusoidal formula achieves this through additive injection. RoPE achieves it through geometric rotation. ALiBi achieves it through attention-score penalties. Each approach encodes the same fundamental information (where tokens sit relative to each other) through a different mathematical channel.

The embedding tensor, enriched with positional information, is now ready for the computation that determines how each token attends to every other token in the sequence. That computation is the subject of the next series, Transformers, which takes this positionally-enriched tensor and constructs the full attention mechanism from first principles.